Programmazione parallela su architetture GPU

Appunti personali

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autori
Emanuele Galiano
Sofia Lo Vecchio

Anno accademico
Anno Accademico 2025/2026

Versione: 30 aprile 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Programmazione parallela su architetture GPU– Appunti personali* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania tenuto dal Prof. Giuseppe Bilotta nell'anno accademico Anno Accademico 2025/2026. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in unico documento.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/GPU-Parallel-Programming`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Programmazione parallela su architetture GPU – Appunti personali*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Sofia Lo Vecchio
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2025/2026

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.
- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano
Sofia Lo Vecchio

Indice

Licenza **v**
Diritti concessi . . . v
Condizioni . . . v
Assenza di garanzia . . . vi
Testo completo della licenza . . . vi

1 Introduzione **1**
1.1 Calcolo parallelo . . . 1
1.2 Limiti del calcolo parallelo . . . 1
1.2.1 Caso ideale di parallelizzazione . . . 1
1.2.2 Legge di Amdahl . . . 2
1.2.3 Legge di Gustafson . . . 3
1.3 Tipi di parallelismo . . . 4
1.3.1 Task-based parallelism . . . 4
1.3.2 Data-based parallelism . . . 5
1.4 Hardware per il calcolo parallelo . . . 6
1.4.1 Shared memory . . . 7
1.4.2 Distributed memory . . . 7
1.4.3 Ibrida . . . 7
1.5 Concorrenza, latenza e banda . . . 8
1.6 Richiesta e offerta di calcolo parallelo . . . 9
1.6.1 Domanda di calcolo parallelo . . . 9
1.6.2 Offerta di calcolo parallelo . . . 9
1.6.3 Videogiochi e GPU . . . 10
1.7 Storia delle GPU . . . 11
1.7.1 Dai primi adattatori video alle prime schede programmabili . . . 11
1.7.2 La transizione verso la grafica 3D . . . 12
1.7.3 Dalla GPU al GPGPU . . . 12
1.8 GPU moderne . . . 12
1.8.1 GPU e CPU: performance e costi . . . 12
1.9 Parallelizzazione . . . 13
1.9.1 Concetto generale di parallelizzazione . . . 13
1.9.2 Algoritmi apparentemente seriali . . . 14
1.9.3 Parallelismo in hardware . . . 14
1.9.4 Parallelismo in software . . . 15

2 Fondamenti di GPGPU **17**
2.1 Architettura di un programma GPGPU . . . 17
2.1.1 Kernel, work-item e work-group . . . 17
2.2 Stream-Processing . . . 18
2.2.1 Kernel e dati . . . 18
2.2.2 Esempio: Mandelbrot fractal . . . 18
2.3 Memoria . . . 20
2.4 Struttura di un programma GPGPU . . . 20
2.4.1 Pipeline di esecuzione . . . 20
2.4.2 Comunicazione host-device . . . 21
2.4.3 Sorgente singolo e separato . . . 21
2.5 Stream Computing . . . 23
2.5.1 Hardware e controller . . . 24
2.5.2 SPMD, SIMD e SIMT . . . 24
2.5.3 Wide cores . . . 25
2.5.4 Strutture dati ideali . . . 25
2.6 Asincronicità . . . 28
2.7 CUDA vs OpenCL . . . 28
2.7.1 Differenze lato host . . . 28
2.7.2 Differenze lato device . . . 29

3 Basi di un programma OpenCL **31**
3.1 Boilerplate OpenCL . . . 31
3.2 Creazione del buffer device . . . 32
3.3 Creazione del kernel . . . 32
3.4 Esecuzione del kernel . . . 33
3.5 Recupero dei dati . . . 35

4 Ottimizzazione e gestione dell'esecuzione in OpenCL **37**
4.1 Mapping al posto di read . . . 37
4.2 Modifica nella creazione del buffer . . . 37
4.3 Recupero dei dati con il mapping . . . 38
4.4 Accesso ai dati . . . 38
4.5 Fase di unmapping . . . 39
4.6 Numeri della griglia di lancio . . . 39
4.6.1 Global size ideale . . . 39
4.6.2 Numeri primi . . . 39
4.7 Aliasing . . . 41
4.8 Vettorizzazione . . . 42
4.8.1 Kernel di inizializzazione . . . 43
4.8.2 Caso base . . . 43
4.8.3 Stride scalare contiguo . . . 43
4.8.4 Stride scalare distribuito . . . 44
4.8.5 Separazione delle fasi di load e store . . . 45
4.8.6 Tipi vettoriali . . . 45

4.8.7 Sliding window . . . 46
4.9 Matrici in OpenCL . . . 47
4.9.1 Rappresentazione in memoria . . . 47
4.9.2 Griglia di lancio 2D . . . 48
4.9.3 Coalescenza in 2D e orientamento del warp . . . 49
4.9.4 Padding (pitch) . . . 50

5 Local memory e sincronizzazione 53
5.1 Il problema: smoothing di un vettore . . . 53
5.2 Versione scalare . . . 53
5.3 Costo dei branch: `if` indipendenti vs `if/else` . . . 54
5.4 Versione vettoriale . . . 55
5.5 La local memory . . . 56
5.6 Indici locali: `get_local_id()` e `get_local_size()` . . . 56
5.7 Trade-off nella dimensione della cache . . . 57
5.8 Race condition in local memory . . . 57
5.9 Barriere . . . 57
5.10 Allocazione di local memory dall'host . . . 58
5.11 Versione con local memory . . . 58
5.12 Versione vettoriale con local memory . . . 59
5.13 Trasposizione di una matrice . . . 61
5.14 Letture coalescenti vs scritture coalescenti . . . 61
5.15 Trasposizione con tile in local memory . . . 62
5.16 Bank conflict nella local memory . . . 64
5.16.1 Soluzione 1: padding . . . 65
5.16.2 Soluzione 2: indirizzamento circolare . . . 66

6 Immagini in OpenCL 69
6.1 Formato dell'immagine . . . 69
6.1.1 Lettura nel kernel . . . 69

Capitolo 1

Introduzione

1.1 Calcolo parallelo

Il calcolo parallelo è un paradigma di programmazione che consente di eseguire più operazioni contemporaneamente, sfruttando la potenza di elaborazione di più unità di calcolo. A differenza dello pseudoparallelismo, che simula l'esecuzione simultanea di più processi su un singolo processore, il calcolo parallelo utilizza effettivamente più processori o core per eseguire compiti in parallelo.

Questo paradigma è utile perché riduce i tempi di esecuzione e aumenta la dimensione dei dati processabili, consentendo di affrontare problemi più complessi e di ottenere risultati più rapidamente.

1.2 Limiti del calcolo parallelo

1.2.1 Caso ideale di parallelizzazione

Per comprendere i limiti del calcolo parallelo è utile partire da un modello ideale. Supponiamo che un programma sia completamente parallelizzabile, cioè che tutto il lavoro possa essere suddiviso tra più unità di calcolo senza alcuna parte sequenziale.

Se T_0 indica il tempo di esecuzione del programma in modalità sequenziale e N il numero di unità di calcolo disponibili, il tempo di esecuzione ideale del programma parallelo sarebbe:

$$T_L(N) = \frac{T_0}{N}$$

In questo caso il tempo di esecuzione diminuisce linearmente all'aumentare del numero di processori. Lo speed-up ottenuto è quindi:

$$S_L(N) = \frac{T_0}{T_L(N)} = N$$

Questo significa che, in condizioni ideali, raddoppiare il numero di unità di calcolo dimezza il tempo di esecuzione. In tale modello non esiste quindi un limite teorico allo speed-up ottenibile aumentando il numero di processori.

1.2.2 Legge di Amdahl

Il calcolo parallelo non può essere applicato a tutti i problemi, e anche quando è possibile, esiste un limite alla velocità di esecuzione che può essere raggiunta. Questo limite è descritto dalla legge di Amdahl:

> *La velocità massima di un programma parallelo è limitata dalla frazione di codice che deve essere eseguita in modo sequenziale.*

Adamahl ha dimostrato che se una frazione p del programma può essere parallelizzata, allora la velocità massima teorica che si può ottenere è data da:

$$T_A(N) = \frac{pT_0}{N} + (1-p)T_0$$

dove T_0 è il tempo di esecuzione del programma sequenziale e N è il numero di processori. Si può anche esprimere il massimo speed-up come:

$$S_A(N) = \frac{T_0}{T_A(N)} = \frac{1}{p/N + (1-p)} = \frac{N}{p + N(1-p)}$$

Da questa formula si evince che, anche con un numero infinito di processori, il massimo speed-up è

$$\lim_{N\to\infty} S_A(N) = \frac{1}{1-p}$$

Il che significa che se una parte significativa del programma è sequenziale, il miglioramento ottenuto con il calcolo parallelo sarà limitato, indipendentemente dal numero di processori utilizzati.

p	2	4	100	200	400	∞
10%	1.05	1.08	1.11	1.11	1.11	1.11
25%	1.14	1.23	1.33	1.33	1.33	1.33
50%	1.33	1.60	1.98	1.99	1.99	2.00
75%	1.60	2.29	3.88	3.94	3.97	4.00
99%	1.98	3.88	50.2	66.9	80.2	100

Tabella 1.1: Speed-up teorico secondo la legge di Amdahl al variare della frazione parallelizzabile p e del numero di processori. Come si può notare all'aumentare delle unità di calcolo, lo speed-up tende a un limite massimo che dipende dalla frazione di codice sequenziale.
Nota: anche nel caso ideale, lo speed up massimo è di 100 volte più veloce.

Ottimizzazione della legge di Amdahl. Ipotizzando che il lavoro totale possa essere diviso in parti multiple $p_1, p_2, \ldots, p_r$ e ognuna abbia uno speed-up $s_1, s_2, \ldots, s_r$, allora lo speed-up totale è dato da:

$$S = \frac{1}{\sum_{i=1}^{r} \frac{p_i}{s_i}}$$

In questo caso ci si potrebbe chiedere se convenga concentrarsi sull'ottimizzazione della parte con lo speed-up maggiore s_i oppure su quella che rappresenta la porzione più grande del programma p_i.

Consideriamo ad esempio un programma suddiviso in due parti, con $p_1 = 0.25$ e $p_2 = 0.75$. Supponiamo inoltre che la prima parte possa essere accelerata con uno speed-up $s_1 = 20$, mentre la seconda con uno speed-up $s_2 = 5$.

Se si ottimizza solo la prima parte si ottiene:

$$S = \frac{1}{\frac{0.25}{20} + 0.75} \approx 1.31$$

Se invece si ottimizza solo la seconda parte:

$$S = \frac{1}{0.25 + \frac{0.75}{5}} = 2.5$$

Infine, ottimizzando entrambe le parti:

$$S = \frac{1}{\frac{0.25}{20} + \frac{0.75}{5}} \approx 6.15$$

Questo esempio mostra che, per ottenere il miglior miglioramento complessivo, conviene concentrarsi sull'ottimizzazione della parte che occupa la porzione maggiore del tempo di esecuzione del programma.

1.2.3 Legge di Gustafson

La legge di Amdahl si concetra su quanto è possibile massimizzare lo speed-up assumendo che il lavoro sia fisso (strong scaling).

La legge di Gustafson, invece, si concentra su quanto è possibile aumentare la dimensione del lavoro mantenendo costante il tempo di esecuzione (weak scaling). Tuttavia, è bene notare che l'aumento della quantità di dati processati non è lineare, ma dipende dall'ordine di complessità dell'algoritmo.

> *Se una frazione p del lavoro può avere uno speed-up di fattore s, allora il lavoro totale che può essere eseguito in un tempo costante è dato da:*
>
> $$S_G = \frac{W_S}{W_0} = 1 - p + sp$$
>
> dove W_S è il lavoro totale eseguito in parallelo, W_0 è il lavoro eseguito in modalità sequenziale.

Esempio. Supponiamo che l'80% del lavoro possa essere parallelizzato con uno speed-up di 20, allora nello stesso tempo di esecuzione del programma sequenziale, è possibile eseguire un lavoro totale di:

$$S_G = 1 - 0.8 + 20 \cdot 0.8 = 16.2$$

Non linearità dello speed-up. La legge di Gustafson mostra che, aumentando la dimensione del lavoro, è possibile ottenere uno speed-up maggiore, anche se la frazione di codice parallelizzabile rimane costante. Tuttavia, è importante notare che lo speed-up non aumenta linearmente con

la dimensione del lavoro, ma dipende dalla frazione di codice parallelizzabile e dallo speed-up ottenuto su quella frazione.

Esempio. Supponiamo che W sia $O(D^3)$ (dove D è la dimensione dei dati). Allora $16\times$ più lavoro corrisponde a un aumento della dimensione dei dati di $\sqrt[3]{16} \approx 2.52$ volte.

1.3 Tipi di parallelismo

Un problema può essere parallelizzato in diversi modi, a seconda della natura del lavoro da svolgere e delle dipendenze tra le operazioni. I principali tipi di parallelismo sono:

- Data-based: le operazioni possono essere eseguite in parallelo su diversi dati, ad esempio l'elaborazione di un array o di una matrice.
- Task-based: le operazioni possono essere eseguite in parallelo su diversi compiti, ad esempio l'esecuzione di più funzioni o processi indipendenti.

1.3.1 Task-based parallelism

Questo tipo di parallelismo si basa sull'esecuzione di compiti indipendenti in parallelo. Ad esempio, se un programma deve eseguire più funzioni che non dipendono l'una dall'altra, queste possono essere eseguite contemporaneamente su processori diversi. Si preferisce questo tipo di parallelismo se:

- Il problema può essere suddiviso in compiti (meglio indipendenti e di dimensione simile).
- Numero di task $\approx$ numero di processori disponibili.
- Le unità che possiedono diversi task possono comunicare in modo semplice.

Esempio: pipeline di elaborazione. Consideriamo un carico di lavoro composto da tre tipi di task applicati a più dataset (A, B, C, ...): la **lettura** dei dati (R), l'**elaborazione** o calcolo (C) e la **scrittura** dei risultati (W). Per semplicità assumiamo tempi di esecuzione fittizi costanti pari a $t_R = 2$ per la lettura, $t_C = 3$ per l'elaborazione e $t_W = 2$ per la scrittura. Nel caso **dipendente** ogni dataset deve rispettare la sequenza $R \to C \to W$, cioè i dati devono essere letti prima di poter essere elaborati e i risultati possono essere scritti solo dopo il completamento del calcolo. Nel caso **indipendente**, invece, i task possono essere schedulati senza vincoli di dipendenza e quindi distribuiti più liberamente tra le unità di calcolo.

Nel caso più semplice, mostrato in Figura 1.1, l'esecuzione avviene su una sola unità di calcolo. Tutti i task vengono quindi eseguiti in modo completamente seriale e ogni dataset attraversa l'intera sequenza di operazioni prima che il successivo possa iniziare. L'ordine di esecuzione diventa quindi $R_A, C_A, W_A, R_B, C_B, W_B, \ldots$, rappresentando il caso base senza alcuna forma di parallelismo.

Introducendo una seconda unità di calcolo e mantenendo le dipendenze tra task (Figura 1.2), è possibile ottenere una parziale sovrapposizione delle operazioni. Ad esempio, mentre una unità legge il dataset successivo, l'altra può elaborare quello precedente. Tuttavia la presenza delle dipendenze impone comunque dei vincoli sull'ordine di esecuzione e genera inevitabilmente alcuni periodi di inattività.

Se i task sono indipendenti (Figura 1.3), lo scheduler può distribuire le operazioni tra le due unità con maggiore libertà. Questo permette di riempire meglio i tempi morti e di ottenere un utilizzo più efficiente delle risorse di calcolo, aumentando quindi il grado di parallelismo effettivamente sfruttabile.

Aumentando il numero di dataset, come mostrato in Figura 1.4, la pipeline tende a stabilizzarsi anche nel caso dipendente. Le unità restano occupate più a lungo e il throughput complessivo migliora, ma il cammino critico imposto dalle dipendenze continua comunque a limitare lo speed-up massimo ottenibile.

Utilizzando tre unità di calcolo con task dipendenti (Figura 1.5) l'aggiunta di risorse non produce necessariamente un'accelerazione proporzionale. Il vincolo $R \rightarrow C \rightarrow W$ per ogni dataset rende infatti difficile mantenere tutte le unità occupate contemporaneamente, causando possibili squilibri nel carico di lavoro.

Nel caso indipendente con tre unità (Figura 1.6), invece, il parallelismo può essere sfruttato in modo più efficace. I task possono essere distribuiti dinamicamente tra le unità e il carico di lavoro risulta più bilanciato, permettendo di avvicinarsi ad un utilizzo più efficiente delle risorse disponibili.

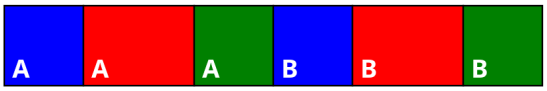

Figura 1.1: Task-based: una sola unità (esecuzione seriale).

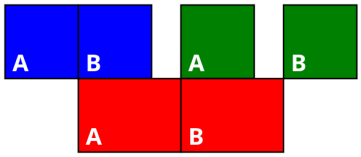

Figura 1.2: Task-based: due unità, caso dipendente.

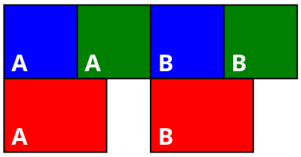

Figura 1.3: Task-based: due unità, caso indipendente.

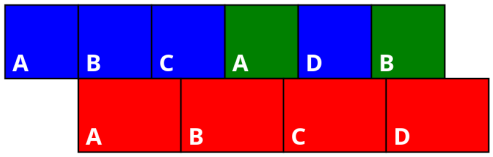

Figura 1.4: Task-based: due unità, più lavoro, caso dipendente.

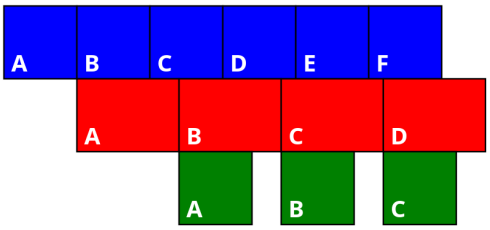

Figura 1.5: Task-based: tre unità, più lavoro, caso dipendente.

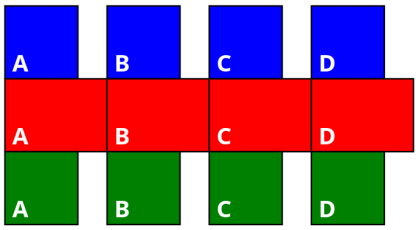

Figura 1.6: Task-based: tre unità, più lavoro, caso indipendente.

1.3.2 Data-based parallelism

Il data-based parallelism si basa sull'esecuzione di operazioni in parallelo su diversi dati. Tutte le unità computazionali fanno lo stesso lavoro, la divisione del lavoro avviene sui dati. Si preferisce

questo approccio quando:

- Il lavoro può essere suddiviso in operazioni simili e semplici da eseguire su diversi dati.
- Si deve lavorare con una grossa mole di dati che può essere facilmente suddivisa in blocchi indipendenti.

Esempio: somma vettoriale. Un esempio classico di parallelismo *data-based* è la somma vettoriale. Consideriamo due vettori A e B di lunghezza N e un vettore risultato C, tale che:

$$C_i = A_i + B_i \qquad i = 0, \ldots, N-1$$

In questo caso l'operazione da eseguire è la stessa per tutti gli elementi, mentre cambiano solo i dati su cui essa viene applicata. Questo rende il problema particolarmente adatto al parallelismo data-based, poiché ogni elemento del vettore può essere elaborato indipendentemente dagli altri.

Seriale. - Nel caso **seriale** (Figura 1.7) un'unica unità di calcolo esegue tutte le operazioni in sequenza. Per ogni indice i vengono letti gli elementi A_i e B_i, viene calcolata la somma e il risultato viene scritto nella posizione corrispondente del vettore C. L'intero vettore viene quindi processato elemento per elemento.

Parallelo. - Nel caso **parallelo** (Figura 1.8), invece, più unità di calcolo eseguono la stessa operazione contemporaneamente su elementi diversi dei vettori. Ogni unità si occupa di un indice differente i, leggendo A_i e B_i, calcolando la somma e scrivendo il risultato in C_i. Poiché non esistono dipendenze tra le operazioni sui diversi elementi, tutte le somme possono essere eseguite in parallelo.

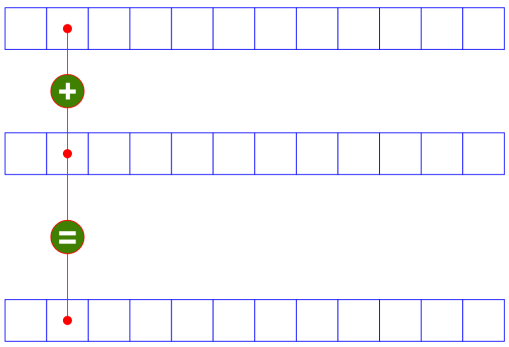

Figura 1.7: Somma vettoriale eseguita in modo seriale.

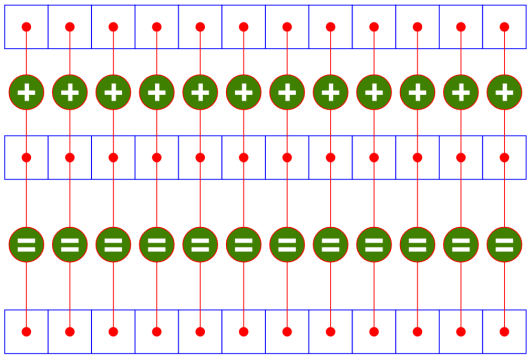

Figura 1.8: Somma vettoriale con parallelismo data-based.

1.4 Hardware per il calcolo parallelo

L'hardware per il calcolo parallelo è progettato per supportare l'esecuzione simultanea di più operazioni. Si può classificare l'hardware parallelo in base a come gestisce le sue risorse di calcolo e, in particolare, di memoria.

1.4.1 Shared memory

In un'architettura a memoria condivisa, tutte le unità di calcolo accedono alla stessa memoria centrale. Questo permette una comunicazione semplice tra le unità, poiché possono leggere e scrivere direttamente sui dati condivisi. Questo modello ha una comunicazione più semplice e meno latenza, ma può essere limitato dalla contesa per l'accesso alla memoria e dalla scalabilità, poiché tutte le unità competono per le stesse risorse di memoria.

1.4.2 Distributed memory

In un'architettura a memoria distribuita, ogni unità di calcolo ha la propria memoria locale e non può accedere direttamente alla memoria delle altre unità. La comunicazione tra le unità avviene attraverso un sistema di messaggistica, in cui i dati devono essere esplicitamente inviati e ricevuti tra le unità. Questo modello è più scalabile e non può essere soggetto a problemi di concorrenza, ma la comunicazione tra le unità è più complessa e può introdurre latenza significativa, soprattutto se i dati devono essere scambiati frequentemente.

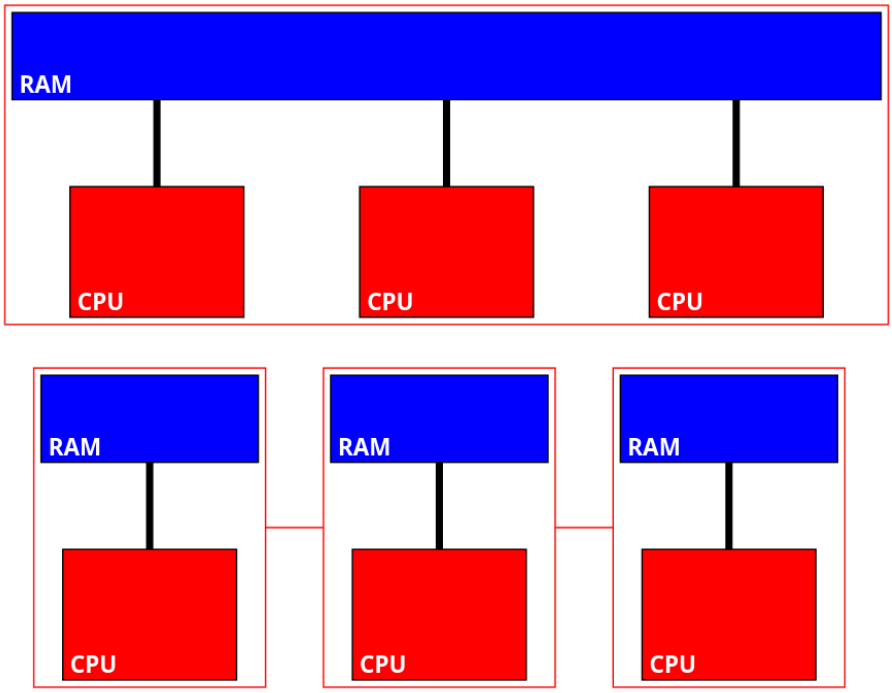

Figura 1.9: Confronto tra architettura a memoria condivisa (in alto), dove più CPU accedono alla stessa RAM, e architettura a memoria distribuita (in basso), dove ogni nodo possiede memoria locale e comunica con gli altri.

1.4.3 Ibrida

Un'architettura ibrida combina elementi di entrambe le architetture a memoria condivisa e distribuita. In questo modello, le unità di calcolo sono organizzate in gruppi, dove ogni gruppo condivide una memoria locale, ma i gruppi stessi comunicano tra loro attraverso un sistema di messaggistica. Questo approccio cerca di bilanciare i vantaggi di entrambi i modelli, offrendo una

maggiore scalabilità rispetto alla memoria condivisa, pur mantenendo una comunicazione più efficiente all'interno dei gruppi.

Un esempio sono i cluster di computer, dove ogni nodo del cluster ha la propria memoria locale, ma i nodi comunicano tra loro attraverso una rete ad alta velocità. All'interno di ogni nodo, le CPU possono accedere alla memoria condivisa, mentre tra i nodi la comunicazione avviene tramite messaggi.

1.5 Concorrenza, latenza e banda

Esistono misure di prestazioni per i sistemi paralleli che aiutano a comprendere l'efficienza e le limitazioni di un'architettura o di un algoritmo parallelo. Le principali sono:

Latenza: la latenza è il tempo necessario per completare un'operazione, ovvero quello che intercorre tra la richiesta e la ricezione di un dato. In un sistema parallelo, la latenza può essere influenzata da fattori come la contesa per le risorse, la comunicazione tra unità di calcolo e la sincronizzazione tra processi. Una bassa latenza è desiderabile per garantire che le operazioni vengano completate rapidamente e che le unità di calcolo non restino inattive in attesa di completare le operazioni. La soluzione principale per minimizzare la latenza è limitare il numero di spostamenti di dati tra le varie unità.

Banda: la banda è la quantità di dati che può essere trasferita in un certo periodo di tempo. In un sistema parallelo, la banda è importante per garantire che i dati possano essere scambiati rapidamente tra le unità di calcolo, soprattutto in architetture a memoria distribuita. Una banda elevata è necessaria per evitare colli di bottiglia nella comunicazione e per garantire che le unità di calcolo possano accedere ai dati di cui hanno bisogno senza ritardi significativi.

Concorrenza: la concorrenza si riferisce alla capacità di un sistema di eseguire più operazioni contemporaneamente. In un sistema parallelo, la concorrenza è essenziale per sfruttare a pieno le risorse di calcolo disponibili. Tuttavia, la concorrenza può introdurre complessità nella gestione delle risorse, nella sincronizzazione tra processi e nella risoluzione di conflitti, soprattutto quando più processi accedono a dati condivisi. Per gestire la concorrenza in modo efficace esistono due soluzioni principali:

- Operazioni atomiche, ovvero operazioni che vengono eseguite in modo indivisibile, garantendo che nessun altro processo possa interferire durante la loro esecuzione.
- Semafori e/o mutex, che sono meccanismi di sincronizzazione che consentono di controllare l'accesso a risorse condivise, evitando conflitti e garantendo la coerenza dei dati.

Esempio: latenza vs banda. Ipotizziamo una connessione ad alta velocità, come *HyperTransport*, con il trasporto fisico di dati tramite un camion pieno di schede di memoria. Nel primo caso la bandwidth è di circa 50 GB/s con una latenza massima di circa 256 ns. Nel secondo caso, considerando un camion carico di schede microSD, la quantità totale di dati trasportabili può essere estremamente elevata, raggiungendo una bandwidth teorica superiore a 1000 TB/s. Tuttavia, il tempo necessario affinché i dati arrivino a destinazione è dell'ordine delle ore, risultando quindi in una latenza molto elevata.

Per migliorare le prestazioni dei sistemi paralleli è quindi importante adottare alcune strategie:

- evitare trasferimenti inutili, in modo da utilizzare la bandwidth in maniera più efficiente;
- accorpare più trasferimenti in operazioni più grandi, così da ammortizzare il costo della latenza.

1.6 Richiesta e offerta di calcolo parallelo

Il calcolo parallelo è diventato sempre più importante negli ultimi anni, sia per la crescente domanda di potenza di calcolo da parte di applicazioni complesse, sia per l'evoluzione dell'hardware che ha reso possibile l'esecuzione di operazioni in parallelo.

1.6.1 Domanda di calcolo parallelo

Generalmente in contesti **scientifici**, **industriali** e **commerciali** si richiede un'elevata potenza di calcolo per affrontare problemi complessi, analizzare grandi quantità di dati e sviluppare applicazioni avanzate. Gli usi principali sono:

- Monitoraggio: il calcolo parallelo consente di analizzare in tempo reale grandi quantità di dati provenienti da sensori, dispositivi IoT e sistemi di monitoraggio, permettendo di rilevare anomalie, prevedere guasti e ottimizzare le prestazioni.
- Simulazione: il calcolo parallelo è essenziale per eseguire simulazioni complesse in campi come la fisica, la chimica, l'ingegneria e la biologia, dove è necessario modellare fenomeni naturali, processi industriali o sistemi complessi.
- Analisi dei dati: il calcolo parallelo è fondamentale per l'analisi di grandi dataset, come quelli generati da social media, e-commerce, finanza e ricerca scientifica, consentendo di estrarre informazioni utili, identificare tendenze e prendere decisioni informate.

1.6.2 Offerta di calcolo parallelo

D'altra parte, l'offerta di calcolo parallelo è cresciuta significativamente grazie all'evoluzione dell'hardware e alla diffusione di tecnologie come i processori multi-core, le GPU e i cluster di computer. Queste tecnologie hanno reso possibile l'esecuzione di operazioni in parallelo su larga scala, offrendo una vasta potenza di calcolo. Oggi è possibile acquistare diverse soluzioni hardware per il calcolo parallelo, ciascuna con caratteristiche, vantaggi e limiti differenti.

Cluster: rappresentano la soluzione più tradizionale su cui si è sviluppato il *distributed computing*. Un cluster è composto da n nodi, ciascuno dotato di CPU multi-core e memoria locale, collegati tra loro tramite una rete ad alta velocità. Questa architettura è ampiamente utilizzata in ambito scientifico e industriale per applicazioni che richiedono elevate capacità di calcolo.

I cluster costituiscono una soluzione classica e consolidata nel tempo, supportata da un vasto ecosistema software e da una grande diffusione di competenze tecniche. Offrono inoltre un'elevata flessibilità, poiché è possibile adattare l'infrastruttura a diverse tipologie di applicazioni.

D'altra parte, presentano anche alcuni svantaggi significativi. L'investimento iniziale in termini di hardware può essere molto elevato e i costi di manutenzione risultano spesso importanti, soprattutto per quanto riguarda il consumo energetico e i sistemi di raffreddamento. Inoltre il rapporto costo/prestazioni può risultare relativamente alto e l'efficienza energetica, misurata come prestazioni per watt consumato, non è sempre ottimale.

GPU: le unità di elaborazione grafica sono nate originariamente per l'elaborazione grafica nei videogiochi, ma nel tempo si è scoperto che la loro architettura altamente parallela poteva essere sfruttata anche per il calcolo generale. Da questa intuizione è nato il paradigma *GPGPU* (*General-Purpose computing on GPU*), che consente di utilizzare le GPU per eseguire calcoli scientifici e applicazioni ad alte prestazioni.

Le GPU offrono diversi vantaggi: il costo iniziale dell'hardware è generalmente inferiore rispetto a soluzioni basate su grandi cluster, così come i costi di manutenzione. Inoltre presentano un rapporto prezzo/prestazioni molto favorevole e un'elevata efficienza energetica, rendendole particolarmente adatte per applicazioni come machine learning, simulazioni numeriche e analisi di grandi quantità di dati.

Tuttavia, si tratta di una tecnologia relativamente più recente rispetto ai cluster tradizionali. Il modello di programmazione GPGPU ha iniziato a diffondersi solo a partire dalla fine del 2006 e dall'inizio del 2007, e per questo motivo l'ecosistema software è storicamente meno maturo. In alcuni contesti sono ancora disponibili meno strumenti, meno applicazioni consolidate e una minore diffusione di competenze specifiche, oltre a una flessibilità talvolta inferiore rispetto alle architetture più tradizionali.

Acceleratori: dispositivi specializzati che affiancano la CPU per eseguire operazioni specifiche in modo altamente parallelo. Questi componenti vengono utilizzati per migliorare le prestazioni in determinati ambiti applicativi, come l'elaborazione di segnali, la crittografia, il machine learning o il calcolo scientifico ad alte prestazioni. Gli acceleratori permettono di delegare alcune operazioni particolarmente intensive a hardware dedicato, aumentando l'efficienza complessiva del sistema.

1.6.3 Videogiochi e GPU

I videogiochi rappresentano un esempio di applicazione che richiede un'elevata potenza di calcolo parallelo, soprattutto per quanto riguarda l'elaborazione grafica. Le GPU sono state sviluppate proprio per soddisfare questa esigenza, consentendo di renderizzare scene complesse in tempo reale. I giochi moderni devono supportare:

- Gameplay più sofisticato
- Grafiche più complesse
- Tre dimensioni
- Fisica più realistica

e per farlo devono sfruttare software e hardware migliori.

Esempio: software migliore. Un esempio storico di ottimizzazione software molto noto nel contesto dei videogiochi è l'algoritmo per il calcolo rapido dell'inversa della radice quadrata,

utilizzato per velocizzare operazioni molto frequenti nella grafica 3D, come la normalizzazione dei vettori.

```
float rsqrt(float x)
{
    float h = x/2;
    int i = *(int*)&x;
    i = 0x5f3758d - (i>>2);
    x = *(float*)&i;
    return x*(1.5f - h*x*x); /* newton */
}
```

Listing 1.1: Implementazione approssimata della radice quadrata inversa

Questo codice implementa una stima rapida di $1/\sqrt{x}$. L'idea è ottenere una prima approssimazione tramite una manipolazione diretta della rappresentazione binaria del numero floating-point e poi migliorarla con un'iterazione del metodo di Newton. L'efficienza di questo approccio deriva dal fatto che evita il calcolo diretto della radice quadrata, che storicamente era molto più costoso dal punto di vista computazionale.

Esempio: hardware migliore. Per ottenere hardware migliore si possono seguire diverse strategie:

- CPU più veloci, con frequenze di clock più elevate e architetture più efficienti. Questo ha un costo elevato e limita la scalabilità, poiché esistono limiti fisici alla velocità di clock e all'efficienza energetica delle CPU.
- Più dischi rigidi, per aumentare la capacità di archiviazione e migliorare le prestazioni di I/O. Tuttavia, questo approccio non migliora direttamente la potenza di calcolo e può introdurre colli di bottiglia nella gestione dei dati.
- Più memoria RAM e cache, per ridurre i tempi di accesso ai dati e migliorare le prestazioni complessive del sistema. Anche questo approccio ha limiti in termini di costo e scalabilità, poiché la memoria è una risorsa costosa e l'efficienza energetica può diminuire con l'aumentare della quantità di memoria.
- Hardware video dedicato, come le GPU, che offrono un'architettura altamente parallela e sono progettate per gestire carichi di lavoro specifici come l'elaborazione grafica. Questo approccio è particolarmente efficace per migliorare le prestazioni nei videogiochi, ma può essere meno flessibile rispetto a soluzioni più generiche come i cluster di CPU.

1.7 Storia delle GPU

1.7.1 Dai primi adattatori video alle prime schede programmabili

L'evoluzione delle GPU può essere interpretata come un progressivo aumento di quattro aspetti fondamentali: **risoluzione**, **numero di colori**, **accelerazione hardware** e **programmabilità**. Le prime schede video erano semplici adattatori grafici: **CGA** (1981) supportava 640×200 in monocromatico oppure 320×200 con 4 colori, oltre a modalità testuali con 16 colori. Con **EGA**

(1984) si passa a 640×350 con una palette di 16 colori scelta da un insieme di 64, ottenendo quindi una migliore qualità visiva sia in termini di dettaglio sia di rappresentazione cromatica.

Un salto più importante si ha con **VGA** (1987), che introduce 6 bit per colore per pixel, fino a 262 144 colori rappresentabili, e modalità standard come 640×480 con 16 colori da una palette di 256 e 320×200 con 256 colori. VGA viene anche considerato il primo adattatore video pienamente programmabile. Nello stesso periodo compaiono soluzioni come **IBM 8514**, che oltre a supportare 1024×768 con 256 colori in modalità interlacciata introducono l'accelerazione hardware di primitive 2D, come il disegno di linee, il riempimento di aree e il *bit blit*. Con **XGA** (1990) si consolidano ulteriormente questi miglioramenti, con modalità come 800×600 a 16 bit di colore e 1024×768 con palette a 256 colori.

1.7.2 La transizione verso la grafica 3D

Negli anni Novanta, la diffusione delle interfacce grafiche e soprattutto dei videogiochi 3D cambia il ruolo delle schede video: non devono più soltanto visualizzare immagini, ma accelerare trasformazioni geometriche, illuminazione e rendering di scene tridimensionali. Nascono così le prime schede dedicate al 3D, che ottengono grande successo commerciale grazie a una grafica più pulita e animazioni più fluide.

Per rendere l'accesso a questo hardware più uniforme si affermano API standard come **OpenGL** e **DirectX**. Con l'introduzione del **programmable shading**, le schede video smettono di essere pipeline fisse e diventano dispositivi progressivamente programmabili: è in questo contesto che si diffonde il termine **GPU**.

1.7.3 Dalla GPU al GPGPU

Da qui nasce anche l'idea di utilizzare la GPU non solo per la grafica ma anche per il calcolo generale. In una prima fase, intorno al 2004, i problemi numerici venivano trasformati artificialmente in problemi grafici e risolti tramite il rendering, in modo complesso ma spesso efficace. La vera svolta arriva quando i produttori rendono accessibile in modo diretto la potenza di calcolo delle GPU: **ATI/AMD** con **CAL** e **NVIDIA** con **CUDA**. Da questo momento la GPU non è più soltanto un acceleratore grafico, ma una vera piattaforma di calcolo parallelo programmabile.

1.8 GPU moderne

Oggi le GPU sono dispositivi estremamente potenti e versatili, utilizzati non solo per la grafica ma anche per una vasta gamma di applicazioni scientifiche, industriali e commerciali. Le GPU moderne sono caratterizzate da un'architettura altamente parallela, si parla di centinaia di multiprocessori ognuno equipaggiato con circa 100 **Processing Unit** (come i CUDA core di NVIDIA), che consentono di eseguire migliaia di thread contemporaneamente. Una processing unit è un'unità di calcolo che esegue operazioni aritmetiche e logiche sui dati, ed è progettata per gestire carichi di lavoro altamente paralleli.

1.8.1 GPU e CPU: performance e costi

Le GPU e le CPU sono progettate per scopi diversi e presentano caratteristiche architetturali differenti, che is riflettono molto sulle prestazioni finali. Le GPU sono ottimizzate per eseguire

un gran numero di operazioni in parallelo, con un'architettura che consente di gestire migliaia di thread contemporaneamente. Le CPU, invece, sono progettate per eseguire operazioni più complesse e meno parallelizzabili, con un numero limitato di core e una maggiore enfasi sulla latenza. Inoltre, mentre le GPU sono indicate per il parallelismo *task-based*, le CPU sono indicate per il parallelismo *data-based*.

Dalle tabelle riportate in Tabella 1.2 si può notare che le GPU offrono un numero significativamente maggiore di unità di elaborazione (MPs) e una maggiore potenza di calcolo in termini di FLOPS rispetto alle CPU. Inoltre, le GPU tendono ad avere una banda di memoria molto più elevata rispetto alle CPU, il che è cruciale per gestire i grandi volumi di dati necessari per le applicazioni di calcolo parallelo. Tuttavia, è importante considerare anche i costi associati all'acquisto e alla manutenzione di questi dispositivi, nonché la complessità dell'ecosistema software e la disponibilità di competenze specifiche per sfruttare a pieno le potenzialità delle GPU.

GPU vs CPU: performance

Model	MPs	PE/MP	FLOPS (TFLOPS)	bandwidth (GB/s)
AMD Radeon VII	60	64	11.1	1028
NVIDIA TITAN RTX	72	64	12.4	672

Model	cores	PE/core	FLOPS (GFLOPS)	bandwidth (GB/s)
AMD EPYC 7702P 2GHz	64	16	4096	143.1
Intel Xeon W-3275M 2.5GHz	28	32	3072	131.13

Tabella 1.2: Confronto prestazionale tra alcune GPU e CPU.

APU. Una terza opzione è rappresentata dalle APU (**Accelerated Processing Unit**), che integrano CPU e GPU su un unico chip. Le APU offrono un compromesso tra le prestazioni di calcolo parallelo delle GPU e la flessibilità delle CPU, consentendo di eseguire sia operazioni generali che specifiche per la grafica in modo efficiente. Tuttavia, le APU tendono ad avere prestazioni inferiori rispetto a soluzioni dedicate come le GPU discrete, ma possono essere una scelta interessante per applicazioni che richiedono un equilibrio tra potenza di calcolo e costi.

1.9 Parallelizzazione

1.9.1 Concetto generale di parallelizzazione

La parallelizzazione è il processo di suddivisione di un problema o di un algoritmo seriale in un algoritmo parallelo. Ipotizziamo due estremi:

Un algoritmo intrinsecamente seriale è un algoritmo che non può essere parallelizzato, poiché vi è una dipendenza tra le operazioni che rende impossibile eseguirle in contemporanea (ad esempio, un'operazione dipende dai risultati delle operazioni precedenti). In questo caso, l'unico modo per eseguire l'algoritmo è in modalità sequenziale, e lo speed-up ottenibile con il calcolo parallelo è limitato. Tuttavia un modo per parallelizzare un algoritmo seriale è quello di fare parallelismo *data-based*, ovvero eseguire in parallelo operazioni simili su dati diversi, anche se l'algoritmo stesso non è parallelizzabile.

Un algoritmo intrinsecamente parallelo è un algoritmo che può essere suddiviso in parti indipendenti che possono essere eseguite contemporaneamente. In questo caso, è possibile ottenere uno speed-up significativo utilizzando più unità di calcolo, poiché le operazioni possono essere eseguite in parallelo senza dipendenze.

1.9.2 Algoritmi apparentemente seriali

Esistono però vie di mezzo, ovvero algoritmi che non sono intrinsecamente seriali ma che possono essere parallelizzati se interpretati in maniera differente. Alcuni esempio sono:

- Riduzione: operazioni che combinano più elementi in un unico risultato, come la somma di un array o il prodotto di matrici. Questi algoritmi possono essere parallelizzati suddividendo i dati in blocchi e combinando i risultati parziali. Tuttavia, la parallelizzazione ha un requisito: è necessario (e sufficiente) che l'operazione sia associativa.

- Scan: operazioni che producono un array di risultati parziali, come la somma cumulativa o il prefisso di un array. Questi algoritmi possono essere parallelizzati utilizzando tecniche come il *divide and conquer* per suddividere i dati e combinare i risultati.

- Algoritmi di ordinamento: algoritmi come il merge sort o il quicksort possono essere parallelizzati suddividendo l'array in parti più piccole e ordinando ciascuna parte in parallelo, per poi combinare i risultati.

1.9.3 Parallelismo in hardware

Dal punto di vista dell'hardware, esistono diversi modelli con cui il parallelismo può essere implementato. I principali sono:

SIMD (*Single Instruction, Multiple Data*) indica un modello in cui una singola istruzione opera contemporaneamente su più dati. Questo approccio è tipico delle istruzioni vettoriali presenti nelle CPU moderne, come MMX, AltiVec, SSE, NEON o AVX, ed è particolarmente efficace quando si devono eseguire gli stessi calcoli su vettori o array.

SPMD (*Single Program, Multiple Data*) indica un modello nel quale la stessa sequenza di istruzioni viene eseguita da più unità di calcolo contemporaneamente su dati differenti. In questo caso non si parla necessariamente di una singola istruzione eseguita nello stesso istante, ma dello stesso programma applicato a porzioni diverse dell'input. Questo modello può essere supportato sia dall'hardware sia dal software.

SIMT (*Single Instruction, Multiple Threads*) è un modello tipico delle GPU. In questo caso l'hardware gestisce automaticamente gruppi di thread che eseguono la stessa istruzione in parallelo. Si tratta quindi di un modello vicino a SPMD, ma con una gestione dei thread fortemente integrata a livello hardware.

In generale, si può osservare che le **GPU** moderne sono tipicamente associate ai modelli **SPMD** e **SIMT**, mentre le **CPU** moderne sono principalmente **SIMD**, eventualmente affiancate da forme di SPMD controllate dal software.

1.9.4 Parallelismo in software

Dal punto di vista software, il parallelismo può essere organizzato secondo modelli differenti. I principali sono:

Multi-processing è il parallelismo basato su più thread in memoria condivisa. In questo caso un singolo programma genera più thread che operano sullo stesso spazio di memoria condiviso. Questo modello è tipico dei sistemi multicore e lo standard più diffuso su CPU è **OpenMP**.

Message-passing è un modello nel quale più processi distinti comunicano tra loro scambiandosi messaggi. Questo approccio è naturale nei sistemi a memoria distribuita, dove ogni processo possiede una propria memoria locale e la cooperazione avviene tramite comunicazione esplicita. Lo standard di riferimento in questo ambito è **MPI** (*Message Passing Interface*).

Hybrid combina i due modelli precedenti: più processi comunicano tramite messaggi e, all'interno di ciascun processo, sono presenti più thread. Questo modello è molto comune nei sistemi HPC moderni, poiché consente di sfruttare contemporaneamente il parallelismo interno ai nodi e quello tra nodi diversi.

Capitolo 2

Fondamenti di GPGPU

2.1 Architettura di un programma GPGPU

L'architettura di un programma GPGPU è composta da tre componenti principali:

Host Il processore centrale (CPU) che esegue il codice principale del programma e gestisce la comunicazione con la GPU.

Device La GPU, che esegue i kernel (flussi di esecuzione parallela) e gestisce la memoria dedicata.

Compute Unit (CU) Un'unità di elaborazione all'interno della GPU che esegue i kernel in parallelo. Ogni CU contiene più core di elaborazione, che possono eseguire kernel in parallelo.

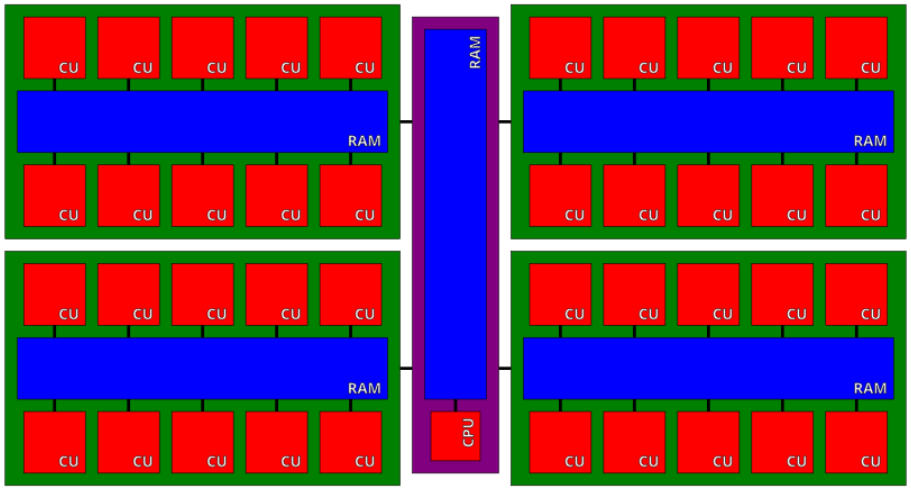

Figura 2.1: Architettura di un sistema OpenCL, con CPU in viola(host) e diverse GPU (device) in verde che comunicano tra loro.

2.1.1 Kernel, work-item e work-group

Un kernel è una sequenza di istruzioni che può essere eseguita in parallelo su più compute unit della GPU. Ogni istanza di esecuzione del kernel è detta **work-item** (in CUDA vengono chiamati "thread") e ogni work-item che appartengono allo stesso **work-group** (un gruppo di work-item) condividono la memoria locale e possono sincronizzarsi tra loro.

La collezione di tutti i work-item che eseguono un kernel è chiamata **launch grid** (o, in OpenCL, "NDRange"). La dimensione del launch grid è definita in termini di numero di work-item e work-group, e può essere configurata per massimizzare l'efficienza dell'esecuzione del kernel sulla GPU.

2.2 Stream-Processing

La prima forma di parallelismo su GPU è lo *stream-processing*, in cui i dati vengono elaborati in flussi (streams) e ogni elemento del flusso viene elaborato in parallelo. Questo modello è adatto per operazioni che possono essere eseguite in modo indipendente su ciascun elemento, come ad esempio l'elaborazione di immagini o la simulazione di particelle, e rientra nella categoria del parallelismo *data-based*.

2.2.1 Kernel e dati

In questo caso il kernel è la sequenza di istruzioni da eseguire su porzioni corrispondenti di più input stream per ottenere uno stream di output. Il kernel lavora solo con i dati che gli sono stati assegnati, e non ha accesso a dati al di fuori del suo work-item o work-group.

In particolare, le operazioni che possono essere eseguite in parallelo sono quelle per cui il risultato delle operazioni è indipendente dalla gestione della divisione e della elaborazione dei dati, processi entrambi gestiti interamente dall'hardware. Quest'ultima caratteristica fa sì che lo stream processing effettuato su determinati dati eseguendo delle date operazioni abbia lo stesso risultato indipendentemente dall'hardware utilizzato (cambia solamente il tempo impiegato).

Il programmatore non ha alcun controllo sulla divisione del lavoro tra i work-item o tra i work-group e non può sincronizzare i work-item tra loro, se non quelli appartenenti allo stesso work-group.

2.2.2 Esempio: Mandelbrot fractal

Il frattale di Mandelbrot è un insieme di punti complessi che non diverge quando iterati attraverso una funzione quadratica. Per generare un'immagine del frattale, ogni pixel dell'immagine può essere calcolato indipendentemente, rendendo questo problema ideale per il parallelismo su GPU. Si può rapresentare la serie di questo frattale come

$$z_{n+1} = z_n^2 + c$$

dove z è un numero complesso e c è un punto complesso che rappresenta la posizione del pixel nell'immagine. Il kernel calcola il numero di iterazioni necessarie affinché z diverga (superi un certo limite) per ogni pixel, e questo numero di iterazioni viene utilizzato per determinare il colore del pixel nell'immagine finale.

Versione seriale. La versione seriale del codice per generare il frattale di Mandelbrot potrebbe essere scritta come:

```
int nrows, ncols, maxiter;
float xmin, xmax, ymin, ymax;
int *bitmap;
```

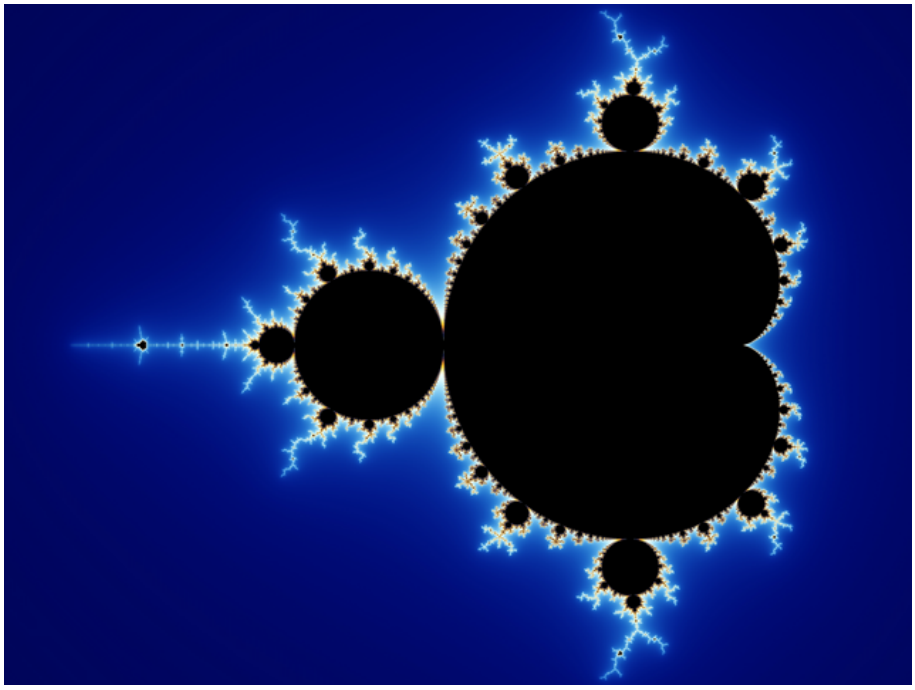

Figura 2.2: Frattale di Mandelbrot generato utilizzando il parallelismo su GPU.

```
for (int row = 0; row < nrows; ++row) {
    for (int col = 0; col < ncols; ++col) {
        complex z, c;
        int iter;

        c.real = xmin + col*(xmax - xmin)/ncols;
        c.img  = ymin + row*(ymax - ymin)/nrows;

        z = c;

        for (iter = 0; iter < maxiter; ++iter) {
            z = add(mul(z, z), c); /* z_{n+1} = z_n^2 + c */
            if (norm(z) > 2)
                break;
        }

        bitmap[row*ncols + col] = !!(iter < maxiter);
    }
}
```

Listing 2.1: Programma seriale per il calcolo dell'insieme di Mandelbrot

Il core parallelizzabile di questo codice è il doppio ciclo annidato che itera su ogni pixel dell'immagine. Ogni iterazione del ciclo interno calcola il numero di iterazioni necessarie affinché z diverga per un pixel specifico, e questo calcolo può essere eseguito in parallelo per tutti i pixel dell'immagine. Questo può essere considerato un candidato ideale per scrivere un kernel che esegue il calcolo in parallelo su una GPU, dove ogni work-item si occupa di calcolare il numero di

iterazioni per un pixel specifico.

2.3 Memoria

I device di calcolo hanno la propria memoria dedicata, che è separata dalla memoria del host. Gli spazi di memoria presenti sulla GPU sono diversi:

Global Memory La memoria globale è accessibile a tutti i work-item e work-group, ma ha una latenza elevata. I contenuti in essi presenti permangono per la durata di esecuzione dell'intero programma e i dati possono essere modificati dai kernel.

Constant Memory La memoria costante è una porzione di memoria globale che è ottimizzata per il broadcasting. Può essere modificata solo dall'host e se necessario può essere cambiata tra l'esecuzione di un kernel e l'altro.

Images (o texture in CUDA) La memoria per le immagini è ottimizzata per l'accesso 2D e 3D, e supporta operazioni di filtraggio, interpolazione, l'indexing multidimensionale e la normalizzazione delle coordinate.

Local Memory (shared memory in CUDA) La memoria locale si trova fisicamente sulle compute unit e, a differenza della memoria globale, è volatile e viene allocata all'inizio di un determinato kernel e successivamente svuotata. Può essere comunque utilizzata dai work-item appartenenti allo stesso work-group per condividere dati e sincronizzarsi tra loro. Viene utilizzata per algoritmi più complessi che richiedono la condivisione di dati tra i work-item, come ad esempio le operazioni di riduzione o di convoluzione, e permette di eseguire su CPU algoritmi non imbarazzantemente paralleli.

Private memory (local memory in CUDA) Questa memoria contiene dati temporanei specifici di un singolo work-item, tipitcamente i dati da inserire all'interno dei registri di esecuzione. Se non vi sono registri sufficienti per la quantità di dati da gestire si ha un *register spill*[1] e i dati vengono spostati in local memory, con conseguente degrado delle prestazioni.

2.4 Struttura di un programma GPGPU

2.4.1 Pipeline di esecuzione

Un programma GPGPU è composto da 4 parti principali:

Selects. - Selezione del dispositivo di calcolo (GPU) su cui eseguire il programma. In questo caso CUDA fa tutto a runtime dietro le quinte, mentre OpenCL richiede al programmatore di specificare esplicitamente il dispositivo da utilizzare. Esempio: se si dispone di più GPU, è possibile selezionare quella più adatta per l'esecuzione del programma, ad esempio in base alla quantità di memoria disponibile o alla potenza di calcolo.

[1] Il register spill è un fenomeno che si verifica quando un kernel utilizza più registri di quelli disponibili sulla GPU, costringendo il compilatore a spostare alcuni dati dalla memoria dei registri alla memoria globale o locale. Questo può causare un degrado delle prestazioni, poiché l'accesso alla memoria globale o locale è molto più lento rispetto all'accesso ai registri.

Uploads. - Trasferimento dei dati dal host alla GPU. Questo passaggio è necessario perché la GPU ha la propria memoria dedicata e non può accedere direttamente alla memoria del host. Esempio: se si sta elaborando un'immagine, è necessario caricare i dati dell'immagine nella memoria della GPU prima di eseguire il kernel che elabora l'immagine.

Runs. - Esecuzione del kernel sulla GPU. In questo passaggio, il kernel viene lanciato sulla GPU e i work-item eseguono il codice in parallelo. Esempio: se si sta calcolando il frattale di Mandelbrot, è necessario lanciare un kernel che esegue il calcolo per ogni pixel dell'immagine in parallelo.

Downloads. - Trasferimento dei dati dalla GPU al host. Dopo l'esecuzione del kernel, i risultati devono essere trasferiti nuovamente al host per essere utilizzati o visualizzati. Esempio: se si sta elaborando un'immagine, è necessario scaricare i dati dell'immagine elaborata dalla memoria della GPU al host per visualizzarla o salvarla su disco.

2.4.2 Comunicazione host-device

Considerando un setup moderno:

- Bandwidth host: 50 GB/s (PCIe 4.0 x16)
- Bandwidth device: 900 GB/s (HBM2)

Il problema principale di questo tipo di architettura è che la comunicazione tra host e device è molto più lenta rispetto alla comunicazione all'interno del device stesso. Pertanto, è importante minimizzare il numero di trasferimenti di dati tra host e device per massimizzare le prestazioni del programma GPGPU. In particolare, il trasferimento dati da host a GPU dipende dalla banda passante del bus PCI express, che si aggira sui 16 GB/s.

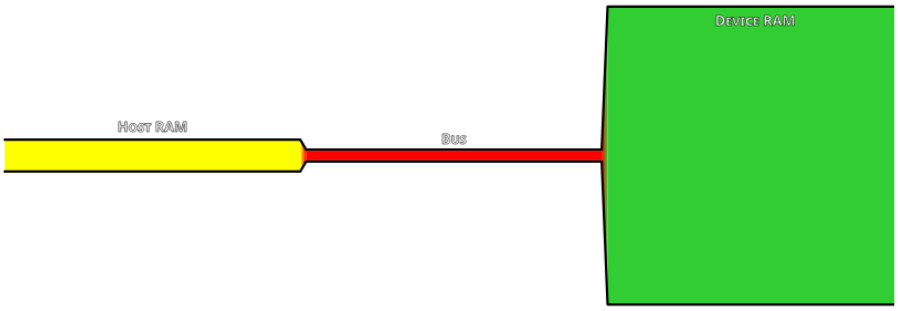

Figura 2.3: Comunicazione tra host e device, con i trasferimenti di dati che avvengono attraverso il bus PCIe.

2.4.3 Sorgente singolo e separato

Come già accennato, un programma GPGPU è composto da due parti distinte: il codice *host*, eseguito sulla CPU, e il codice *device*, costituito dai kernel che verranno eseguiti sulla GPU. Queste due componenti non solo vengono eseguite su architetture differenti, ma sono anche tradotte in linguaggi macchina distinti; inoltre, tale linguaggio può variare anche tra diverse generazioni di GPU dello stesso produttore.

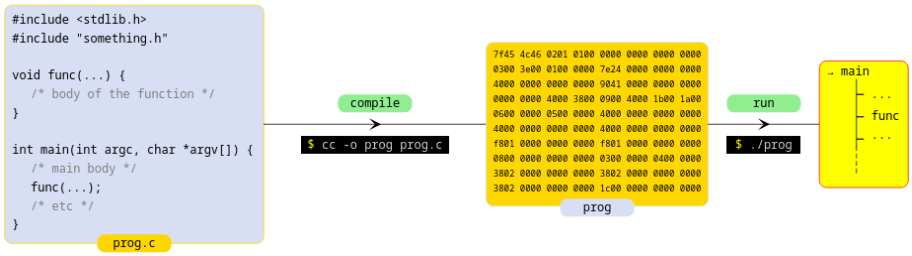

Figura 2.4: Processo di compilazione ed esecuzione di un programma C: dal codice sorgente alla generazione dell'eseguibile e alla sua esecuzione.

Il modo in cui queste due parti vengono organizzate e compilate porta a distinguere due approcci principali: il modello *single-source* e il modello *separate-source*.

Single-source. Nel modello *single-source*, tipico ad esempio di CUDA, il codice host e quello device convivono all'interno dello stesso file sorgente. In questo caso è il compilatore a occuparsi automaticamente di separare le due componenti e di gestirne la compilazione.

Più precisamente, le funzioni destinate all'esecuzione sulla GPU vengono identificate tramite annotazioni specifiche, come `__global__`. Il compilatore (ad esempio `nvcc`) analizza il sorgente e lo suddivide in una parte host, che viene passata al compilatore tradizionale, e una parte device, che viene compilata con un backend dedicato alla GPU. Il risultato è la produzione di due codici macchina distinti, uno per la CPU e uno per il device.

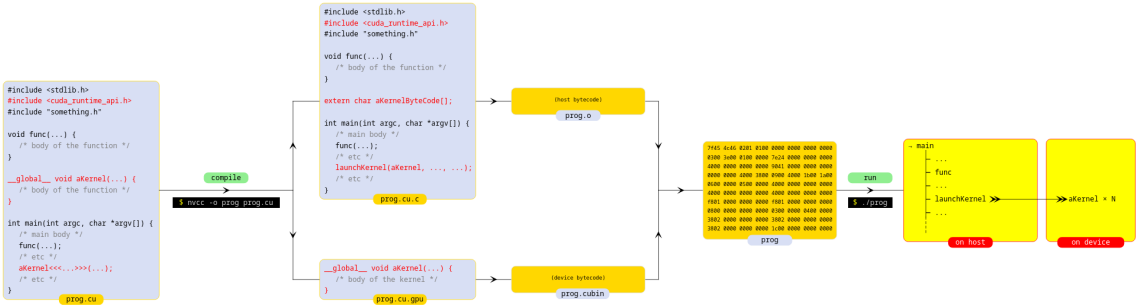

Figura 2.5: Processo di compilazione ed esecuzione in CUDA: il codice single-source viene separato in componente host e device, compilato in bytecode distinti e ricombinato in un unico eseguibile che coordina CPU e GPU.

Dal punto di vista dell'esecuzione, il codice host può invocare un kernel tramite una sintassi dedicata, come `aKernel«<...»>();`, specificando la configurazione della griglia di esecuzione. È importante osservare che il lancio del kernel è asincrono rispetto all'host: questo significa che l'host continua la propria esecuzione senza attendere il completamento del kernel, e quindi una misura del tempo attorno al lancio non rappresenta il tempo reale di esecuzione sul device.

Questo approccio risulta generalmente più semplice da utilizzare, in quanto riduce la quantità di codice da gestire e garantisce coerenza tra i tipi di dato utilizzati lato host e lato device. Tuttavia,

introduce alcuni vincoli: richiede l'uso di un compilatore specifico, che deve essere compatibile con il compilatore host, e impone di specificare a priori il device di destinazione. Inoltre, non consente una compilazione selettiva dei singoli kernel.

Separate-source. Nel modello *separate-source*, adottato ad esempio in OpenCL, il codice device è completamente separato da quello host e viene gestito come entità indipendente.

In questo caso, i kernel risiedono tipicamente in file distinti oppure vengono rappresentati come stringhe all'interno del codice host. Durante l'esecuzione, è il programma host a caricare il sorgente dei kernel e a richiederne la compilazione al driver della GPU. La compilazione avviene quindi a runtime, e non in fase di build, rendendo questo approccio più dinamico e flessibile. I driver, per ottimizzare le prestazioni, mantengono generalmente una cache dei kernel già compilati, evitando di ripetere operazioni costose.

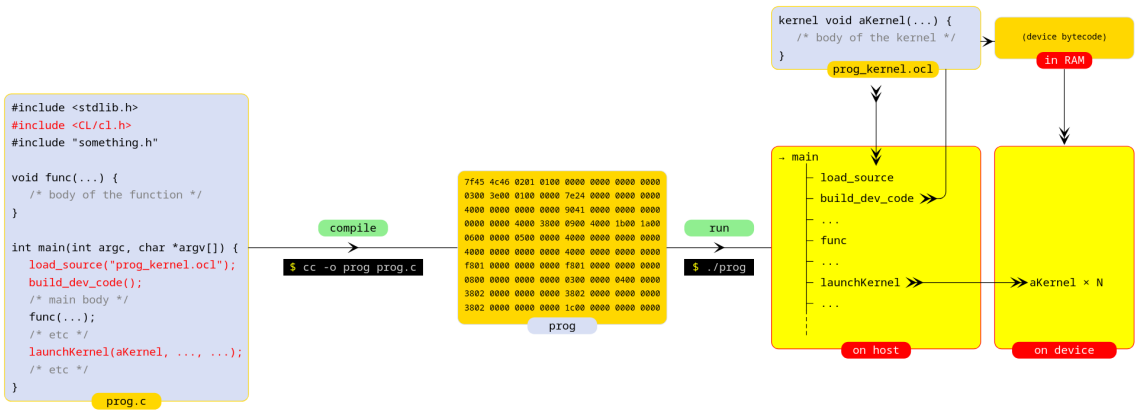

Figura 2.6: Processo di compilazione ed esecuzione in OpenCL: il codice host viene compilato separatamente, mentre i kernel device sono caricati e compilati a runtime dal driver prima dell'esecuzione sulla GPU.

Questo modello offre una maggiore portabilità, in quanto lo stesso codice device può essere compilato su architetture differenti e non richiede l'uso di un compilatore specifico per l'host. Inoltre, permette di selezionare dinamicamente quali kernel compilare ed eseguire.

D'altra parte, questa flessibilità comporta un aumento della complessità: il codice da gestire è maggiore e la separazione tra host e device rende meno immediata la condivisione dei tipi di dato, che devono quindi essere definiti con particolare attenzione, privilegiando strutture semplici e compatibili tra i due ambienti.

2.5 Stream Computing

Lo stream computing è un paradigma di programmazione che si basa sull'elaborazione di flussi di dati in tempo reale. In questo modello, i dati vengono elaborati in modo continuo e parallelo, consentendo di ottenere prestazioni elevate e bassa latenza.

2.5.1 Hardware e controller

Le GPU sono **progettate** per lo stream computing. Una singola compute unit può gestire centinaia di work-item che eseugono lo stesso kernel su più dati in parallelo. La memoria globale, inoltre, ha un'ampia banda passante ma anche alta latenza, per questo motivo si devono usare dei pattern di accesso ottimali per massimizzare le prestazioni. Le GPU sono anche asincrone rispetto alla CPU, il che significa che è possibile eseguire più kernel in parallelo e sovrapporre l'esecuzione dei kernel con i trasferimenti di dati tra host e device.

Il controller di una GPU è responsabile della gestione dell'esecuzione dei kernel e della comunicazione tra host e device. Il controller gestisce la coda dei kernel da eseguire, assegna i work-item alle compute unit e gestisce i trasferimenti di dati tra host e device. Generalmente è un dispositivo ad **alta latenza** e questo influenza le strutture dati utilizzabile che devono essere efficienti da accedere, ha però un'**alta banda passante** che consente di trasferire grandi quantità di dati in parallelo.

2.5.2 SPMD, SIMD e SIMT

Nel contesto del calcolo parallelo su GPU, è importante distinguere tra diversi modelli di esecuzione che descrivono come le operazioni vengono applicate ai dati. I paradigmi più rilevanti sono *SIMD*, *SPMD* e *SIMT*, che, pur essendo correlati, rappresentano livelli di astrazione differenti.

SIMD Il paradigma *Single Instruction Multiple Data* (SIMD) è tipico delle CPU moderne. In questo modello, una singola istruzione viene applicata simultaneamente a più dati organizzati in vettori. Il programmatore deve esplicitamente utilizzare tipi vettoriali (ad esempio `float4`, `float8`, ecc.) e scrivere codice che sfrutti direttamente la larghezza delle unità SIMD.

Questo approccio richiede una certa conoscenza dell'hardware sottostante: per sfruttare pienamente le prestazioni, è necessario adattare il codice alla larghezza dei registri vettoriali (ad esempio SSE, AVX, AVX2). Di conseguenza, il codice risulta meno portabile e più difficile da mantenere.

SPMD Il paradigma *Single Program Multiple Data* (SPMD) rappresenta un modello di programmazione più astratto. In questo caso, il programmatore scrive un singolo programma (il kernel), che viene eseguito molte volte in parallelo su dati diversi. Ogni istanza di esecuzione (work-item) esegue lo stesso codice, ma opera su porzioni differenti dei dati.

Questo modello è particolarmente adatto alla programmazione su GPU, in quanto consente di esprimere facilmente il parallelismo senza preoccuparsi direttamente della gestione delle unità vettoriali.

SIMT Dal punto di vista hardware, le GPU implementano un modello chiamato *Single Instruction Multiple Threads* (SIMT), che può essere visto come una realizzazione efficiente del modello SPMD. Il programmatore scrive codice come se ogni work-item fosse indipendente, ma l'hardware esegue gruppi di work-item (chiamati *warp* in NVIDIA o *wavefront* in AMD) in maniera sincronizzata, condividendo lo stesso flusso di istruzioni.

Questo significa che, anche se il codice sembra scalare (operare su singoli dati), viene in realtà eseguito in modo simile al SIMD, ma con una gestione trasparente da parte dell'hardware. Tuttavia, il controllo di flusso è limitato: se i thread all'interno dello stesso gruppo seguono

percorsi diversi (branch divergenti), le istruzioni vengono serializzate, con una perdita di prestazioni.

Esempio La differenza tra SIMD e SPMD/SIMT può essere illustrata con un semplice esempio di somma tra vettori.

Nel caso SIMD (tipico su CPU), il programmatore lavora esplicitamente su vettori:

```
float4 a, b, c;
a = /* load vector */;
b = /* load vector */;
c = a + b;
```

Nel caso SPMD/SIMT (tipico su GPU), il programmatore scrive invece codice scalare, che verrà eseguito in parallelo su molti elementi:

```
float a, b, c;
a = /* load scalar */;
b = /* load scalar */;
c = a + b;
```

In questo secondo caso, è l'hardware della GPU a replicare automaticamente l'esecuzione su molti dati e a distribuirla tra le unità di calcolo disponibili.

2.5.3 Wide cores

Le GPU sono progettate con un gran numero di core, ma ciascun core è relativamente semplice rispetto a quelli di una CPU. Questo approccio architetturale privilegia il throughput complessivo rispetto alla complessità del singolo core, rendendo le GPU particolarmente efficienti per carichi di lavoro altamente paralleli. Più precisamente, un multiprocessore della GPU (Compute Unit o Streaming Multiprocessor) contiene numerosi *processing elements* (PE), che possono essere visti come unità di calcolo elementari. Tuttavia, questi elementi non operano in modo completamente indipendente: essi eseguono lo stesso flusso di istruzioni in maniera coordinata, secondo un modello simile al SIMD, ma con una gestione demandata all'hardware.

Questa organizzazione introduce alcune limitazioni sul controllo di flusso. In particolare, le GPU sono ottimizzate per schemi di esecuzione regolari, dove tutti i work-item seguono lo stesso percorso. Costrutti complessi come branching irregolare o ricorsione sono poco efficienti o addirittura non supportati. Per gestire le divergenze, l'hardware utilizza tecniche come la *predication* e il *lane masking*, che permettono di eseguire comunque le istruzioni, ma disabilitando selettivamente alcuni work-item, con un possibile degrado delle prestazioni.

Un aspetto fondamentale è che i work-item non vengono eseguiti singolarmente, ma in gruppi a dimensione fissa. Questi gruppi sono chiamati *warp* nell'architettura NVIDIA e *wavefront* in quella AMD, e rappresentano la larghezza effettiva del parallelismo hardware (SIMT width). I work-item all'interno dello stesso gruppo vengono eseguiti in *lock-step*, ovvero condividono lo stesso contatore di programma e avanzano insieme nell'esecuzione.

2.5.4 Strutture dati ideali

L'efficienza di un programma GPGPU dipende in maniera significativa da come i dati sono organizzati in memoria. A differenza delle CPU, dove il costo dell'accesso alla memoria è in parte

nascosto da cache sofisticate, nelle GPU è fondamentale strutturare i dati in modo da sfruttare al meglio la banda disponibile.

Tipologie di dati. Un primo aspetto riguarda la scelta dei tipi di dato. Le GPU sono ottimizzate per operare su dati di dimensione ben definita, tipicamente 32, 64 o 128 bit. Per questo motivo, è spesso conveniente utilizzare tipi vettoriali (come `float2`, `float4`, ecc.), che permettono di allineare meglio i dati e sfruttare in modo più efficiente le operazioni di lettura e scrittura.

Layout in memoria. Un secondo aspetto fondamentale è il layout in memoria. I dati dovrebbero essere disposti in modo contiguo e correttamente allineato: in particolare, l'indirizzo di un dato dovrebbe essere un multiplo della sua dimensione. Ad esempio, un valore `float` (4 byte) viene letto in modo efficiente se il suo indirizzo è multiplo di 4. Un accesso non allineato può richiedere più transazioni di memoria, con un conseguente peggioramento delle prestazioni.

Accessi coalescenti. Infine, è importante considerare che gli accessi alla memoria non avvengono a livello di singolo work-item, ma a livello di gruppi di esecuzione (warp o wavefront). I work-item all'interno dello stesso gruppo eseguono la stessa istruzione in parallelo e, idealmente, accedono a posizioni contigue di memoria. In questo caso, il controller di memoria può soddisfare tutte le richieste con una singola transazione, massimizzando la banda passante. Al contrario, accessi irregolari o non contigui portano a un aumento del numero di transazioni e quindi a una riduzione delle prestazioni.

Esempi di accesso alla memoria

Un errore comune consiste nell'utilizzare strutture dati che sono naturali dal punto di vista logico, ma inefficienti dal punto di vista dell'accesso alla memoria. Consideriamo ad esempio una simulazione di particelle:

```
typedef struct particle {
    float3 pos;
    float3 vel;
} particle_t;

kernel void iteration(global particle_t *particles)
{
    int workitem = get_global_id(0);
    float3 pos = particles[workitem].pos;
    float3 vel = particles[workitem].vel;
    /* do stuff */
}
```

In questo caso, ogni work-item accede ai campi `pos` e `vel` della propria particella. Tuttavia, i dati in memoria sono interleavati (Array of Structures), e quindi i work-item di uno stesso warp non accedono a locazioni contigue per ciascun campo. Questo porta a accessi non coalescenti e a un aumento del numero di transazioni di memoria.

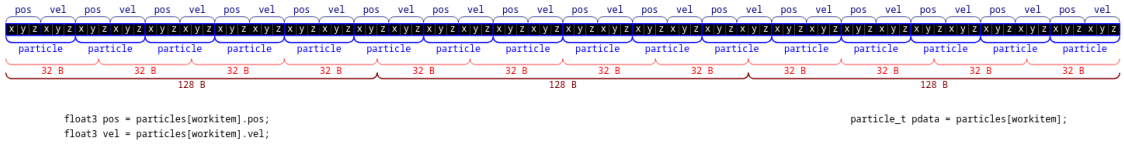

Figura 2.7: Accesso inefficiente con layout Array of Structures: i campi `pos` e `vel` sono interleavati, causando accessi non coalescenti e multiple transazioni di memoria per warp.

Caso inefficiente (Array of Structures) Nel caso mostrato, i work-item accedono a campi distribuiti in memoria in modo non contiguo. Il controller di memoria non può aggregare le richieste e deve effettuare più transazioni, con conseguente perdita di prestazioni.

Caso intermedio (allineamento con float4) Un miglioramento consiste nell'utilizzare tipi allineati, ad esempio:

```
typedef struct particle {
    float4 pos;
    float4 vel;
} particle_t;
```

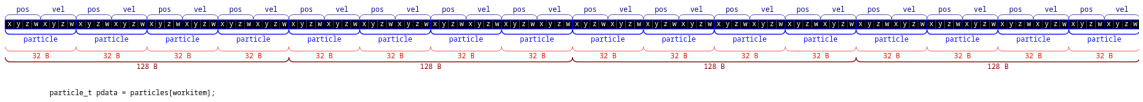

Figura 2.8: Allineamento tramite `float4`: i dati sono meglio allineati e possono essere caricati con meno transazioni, ma il layout rimane di tipo AoS e non è ottimale.

In questo modo, gli accessi sono più regolari e meglio allineati. Tuttavia, i dati restano organizzati come array di strutture, quindi non si ottiene ancora il massimo livello di efficienza.

Caso efficiente (Structure of Arrays) L'approccio più efficiente consiste nel separare i dati in più array (Structure of Arrays):

```
kernel void iteration(
    global float4 * restrict posArray,
    global float4 * restrict velArray)
{
    int workitem = get_global_id(0);
    float4 pos = posArray[workitem];
    float4 vel = velArray[workitem];
    /* do stuff */
}
```

In questo caso, i work-item di uno stesso warp accedono a elementi contigui degli array `posArray` e `velArray`. Il controller di memoria può quindi servire tutte le richieste con un numero minimo di transazioni (tipicamente una per array), massimizzando la banda passante.

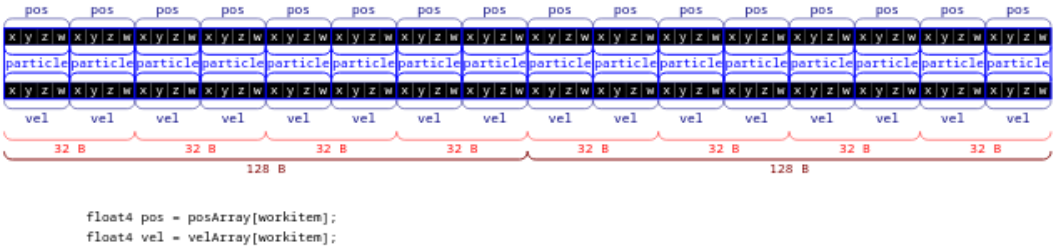

Figura 2.9: Layout Structure of Arrays: i work-item accedono a dati contigui, permettendo accessi coalescenti e riducendo il numero di transazioni di memoria.

2.6 Asincronicità

La GPU opera in maniera indipendente rispetto alla CPU, rendendo la comunicazione tra host e device intrinsecamente asincrona. Per gestire questa interazione, si utilizzano delle code di esecuzione chiamate *command queues* in OpenCL e *streams* in CUDA.

Mentre in CUDA la gestione può essere implicita, in OpenCL la gestione della coda deve sempre essere esplicitata. Per coordinare le operazioni vengono utilizzati gli *eventi*, marcatori associati ai singoli comandi che permettono all'host di monitorare l'avanzamento dei singoli task. Nello specifico, l'host può:

- Sospendere l'esecuzione di un comando fino al completamento di un altro task precedentemente avviato;
- Richiedere lo stato di un evento;
- Imporre delle barriere di sincronizzazione affinché un device completi tutti i comandi pendenti prima di procedere.

Il principale vantaggio dell'asincronicità risiede nella capacità di "nascondere" la latenza dei trasferimenti dati attraverso il bus PCIe sovrapponendo temporalmente le operazioni di I/O con l'esecuzione dei kernel. Inoltre, gli eventi permettono una sincronizzazione granulare (sul singolo kernel) rispetto alla sincronizzazione globale dell'intero device.

2.7 CUDA vs OpenCL

CUDA e OpenCL rappresentano due approcci differenti alla programmazione su GPU. Pur condividendo molti concetti di base (kernel, memoria device, esecuzione parallela), si differenziano sia per il modello di programmazione lato host sia per il linguaggio utilizzato lato device.

2.7.1 Differenze lato host

Dal punto di vista dell'API host, CUDA è una piattaforma proprietaria sviluppata da NVIDIA e progettata per sfruttare al massimo le caratteristiche delle GPU di questo produttore. Questo si traduce in un modello più semplice e diretto: molte operazioni, come la gestione dei device e delle code di esecuzione, sono gestite implicitamente dal runtime. Inoltre, quando si alloca memoria sul device, viene restituito direttamente un puntatore utilizzabile nel codice CUDA.

Categoria	CUDA	OpenCL
Host API		
Supporto hardware	Solo GPU NVIDIA	Multi-vendor (CPU, GPU, FPGA)
Modello sorgente	Single-source	Separate-source
Compilazione kernel	Compile-time	Runtime (driver)
Gestione device	Implicita	Esplicita (platform, device, context)
Gestione code	Implicita	Esplicita (command queue)
Memoria device	Puntatori diretti	Buffer/oggetti astratti
Lancio kernel	Grid/block	NDRange (work-item/work-group)
Device Language		
Linguaggio	C++ esteso	OpenCL C
Libreria standard	Limitata	Completa (built-in)
Swizzling vettori	Non supportato	Supportato (`f.xy`, ecc.)
Tipi vettoriali	Supportati	Supportati
Accesso agli indici	Manuale (`threadIdx`)	Built-in (`get_global_id`)

Tabella 2.1: Confronto tra CUDA e OpenCL.

OpenCL, al contrario, è uno standard aperto pensato per sistemi eterogenei. Questo comporta una maggiore flessibilità, ma anche una maggiore complessità. Il programmatore deve esplicitamente gestire piattaforme, device e code di esecuzione. Inoltre, la memoria allocata sul device non è accessibile tramite puntatori diretti, ma tramite oggetti astratti (buffer), che vengono gestiti attraverso l'API.

Un'altra differenza riguarda l'organizzazione del codice: CUDA adotta un modello *single-source*, mentre OpenCL utilizza un approccio *separate-source*, in cui i kernel vengono compilati a runtime.

2.7.2 Differenze lato device

Per quanto riguarda il linguaggio device, CUDA utilizza un'estensione del C++, mentre OpenCL definisce un proprio linguaggio basato sul C.

Nel caso di CUDA, il linguaggio è fortemente integrato con il modello di esecuzione, ma presenta alcune limitazioni: ad esempio, non esiste una vera libreria standard completa e non sono supportate alcune operazioni tipiche dei linguaggi vettoriali, come lo *swizzling*. Inoltre, l'indice globale di un thread deve essere calcolato manualmente (combinando gli indici di blocco e thread) e la divisione della griglia di lancio in work group deve essere anch'essa effettuata manualmente.

OpenCL, invece, fornisce un linguaggio più ricco dal punto di vista delle operazioni vettoriali, offrendo costrutti come lo swizzling (ad esempio `f.xy`, `f.wwxx`), ed è dotato di funzioni built-in per ottenere automaticamente gli indici dei work-item e dei work-group. Inoltre, OpenCL possiede un meccanismo di estensioni che permettono di sfruttare le caratteristiche architetturali proprie di specifici hardware.

Capitolo 3

Basi di un programma OpenCL

In questo capitolo si spiegano le basi per utilizzare OpenCL seguendo un codice `vecinit.c` per l'allocazione di un vettore e l'inizializzazione dei suoi elementi al loro indice. Il codice completo è disponibile nella cartella `codes/vecinit`.

3.1 Boilerplate OpenCL

La parte iniziale di OpenCL è sempre la stessa, e consiste nella selezione del dispositivo, nella creazione del contesto e della coda dei comandi. Il codice seguente mostra come fare tutto questo:

```
cl_platform_id p = select_platform();
cl_device_id d = select_device(p);
cl_context ctx = create_context(p, d);
cl_command_queue que = create_queue(ctx, d);

cl_program prog = create_program("vecinit.ocl", ctx, d);
```

Listing 3.1: Boilerplate OpenCL

Descrizione del codice

Le varie righe possono essere descritte così:

- `select_platform()` è una funzione che restituisce la piattaforma OpenCL da utilizzare. In un sistema con più piattaforme, è necessario scegliere quella corretta.
- `select_device(p)` è una funzione che restituisce il dispositivo OpenCL da utilizzare. In un sistema con più dispositivi, è necessario scegliere quello corretto.
- `create_context(p, d)` è una funzione che crea un contesto OpenCL per la piattaforma e il dispositivo selezionati. Il contesto è un ambiente in cui vengono eseguiti i kernel OpenCL.
- `create_queue(ctx, d)` è una funzione che crea una coda dei comandi OpenCL per il contesto e il dispositivo selezionati. La coda dei comandi è utilizzata per inviare i comandi al dispositivo.
- `create_program("vecinit.ocl", ctx, d)` è una funzione che crea un programma OpenCL a partire da un file sorgente. Il programma contiene i kernel OpenCL che verranno

eseguiti sul dispositivo. In questo caso, il file sorgente è `vecinit.ocl`, che contiene il kernel per inizializzare un vettore.

3.2 Creazione del buffer device

Il buffer è una regione di memoria allocata sul dispositivo che può essere utilizzata per scambiare dati tra il host e il device. Per creare un buffer, è necessario specificare la dimensione del buffer, le opzioni di accesso, il puntatore ai dati (che generalmente è `NULL` se non si vuole usare un puntatore host) e un puntatore per un eventuale codice di errore. Il codice seguente mostra come creare un buffer:

```
const size_t memsize = sizeof(int)*nels;
cl_int err;
cl_mem d_vec = clCreateBuffer(ctx, CL_MEM_WRITE_ONLY, memsize, NULL, &err);
ocl_check(err, "clCreateBuffer fallito");
```

Listing 3.2: Creazione di un buffer OpenCL

Descrizione del codice

Le varie righe possono essere descritte così:

- `memsize` rappresenta la dimensione del buffer espressa in byte. In questo caso, il buffer contiene `n` elementi di tipo `int`, quindi la dimensione totale è pari a `sizeof(int) * n`.
- `clCreateBuffer(ctx, CL_MEM_WRITE_ONLY, memsize, NULL, &err)` è la funzione che crea il buffer OpenCL. I parametri sono:
  - `ctx` è il contesto OpenCL in cui viene creato il buffer.
  - `CL_MEM_WRITE_ONLY` è un flag che specifica i permessi di accesso al buffer dal punto di vista del kernel. In particolare, indica che il buffer può essere scritto dal kernel, ma non letto. Altri flag comuni sono `CL_MEM_READ_ONLY` e `CL_MEM_READ_WRITE`.
  - `memsize` indica la dimensione del buffer in byte.
  - `NULL` indica che non viene fornito alcun puntatore a memoria host iniziale. L'allocazione del buffer è quindi completamente gestita dal runtime OpenCL.
  - `&err` è un puntatore a una variabile in cui viene salvato il codice di errore restituito dalla funzione. Se la creazione del buffer ha successo, `err` assume valore `CL_SUCCESS`.
- `ocl_check(err, "clCreateBuffer fallito")` è una funzione di utilità che verifica il valore di `err`. Se il codice di errore è diverso da `CL_SUCCESS`, l'errore viene segnalato e l'esecuzione del programma viene interrotta.

3.3 Creazione del kernel

Il kernel è la funzione che viene eseguita sul dispositivo. Per creare un kernel, è necessario specificare il nome del kernel e il programma da cui è stato creato. Il codice seguente mostra come creare un kernel:

```
const char *kname = "init_k";
cl_kernel init_k = clCreateKernel(prog, kname, &err);
ocl_check(err, "clCreateKernel %s fallito", kname);
```

Listing 3.3: Creazione di un kernel OpenCL

Descrizione del codice

Le varie righe possono essere descritte così:

- `kname` è una stringa che contiene il nome del kernel da creare. In questo caso, il kernel si chiama `init_k`.
- `clCreateKernel(prog, kname, &err)` è la funzione che crea il kernel OpenCL. I parametri sono:
  - `prog` è il programma OpenCL da cui viene creato il kernel. Il programma deve essere stato compilato con successo prima di poter creare un kernel da esso.
  - `kname` è il nome del kernel da creare. Questo nome deve corrispondere a una funzione definita nel file sorgente del programma OpenCL.
  - `&err` è un puntatore a una variabile in cui viene salvato il codice di errore restituito dalla funzione. Se la creazione del kernel ha successo, `err` assume valore `CL_SUCCESS`.
- `ocl_check(err, "clCreateKernel %s fallito", kname)` è la solita funzione di errore che verifica il valore di `err` e segnala eventuali errori nella creazione del kernel.

3.4 Esecuzione del kernel

Nella funzione `init` del sorgente C viene assegnato ad ogni elemento del vettore il suo indice. In questa funzione viene eseguito il kernel `init_k` con un certo numero di work-item. Il codice seguente mostra come eseguire il kernel:

```
cl_int err;
cl_event init_evt;

const size_t gws[1] = { n };

cl_uint arg_index = 0;
err = clSetKernelArg(init_k, arg_index, sizeof(cl_mem), &vec);
ocl_check(err, "init_k set kernel arg %u", arg_index);

err = clEnqueueNDRangeKernel(que, init_k,
  1,    /* dimensionalita' griglia: nel nostro caso 1D */
  NULL, /* global work offset */
  gws,  /* global work size: quanti elementi totali lanciare */
  NULL, /* local work size: come raggruppare gli elementi */
  0, NULL, /* wait list: non aspettiamo nulla */
  &init_evt /* evento associato al lancio del kernel */);
```

```
ocl_check(err, "enqueue kernel init_k");
```

Listing 3.4: Esecuzione di un kernel OpenCL

Descrizione del codice

Le varie righe possono essere descritte così:

- `cl_event init_evt` è una variabile di tipo `cl_event` utilizzata per tenere traccia dell'esecuzione di un comando sulla command queue. In questo caso, `init_evt` sarà associato al lancio del kernel `init_k` e potrà essere usato successivamente per sincronizzazione o misure temporali.

- `gws` (global work size) è un array che contiene la dimensione globale del dominio di esecuzione del kernel. Nel caso monodimensionale, `gws[0]` rappresenta il numero totale di work-item. In questo caso vale `n`.

- `clSetKernelArg(init_k, arg_index, sizeof(cl_mem), &vec)` è la funzione che imposta un argomento del kernel. I parametri sono:

  - `init_k` è il kernel su cui si vuole impostare l'argomento.
  - `arg_index` è l'indice dell'argomento. Gli argomenti sono indicizzati a partire da 0.
  - `sizeof(cl_mem)` è la dimensione dell'argomento in byte.
  - `&vec` è un puntatore al valore dell'argomento. In questo caso, `vec` è un buffer OpenCL passato al kernel.

- `clEnqueueNDRangeKernel(que, init_k, 1, NULL, gws, NULL, 0, NULL, &init_evt)` è la funzione che accoda il kernel per l'esecuzione sulla command queue. I parametri sono:

  - `que` è la command queue associata al dispositivo su cui verrà eseguito il kernel.
  - `init_k` è il kernel da eseguire.
  - `1` indica che il dominio di esecuzione è monodimensionale.
  - `NULL` è il global work offset. Non essendo specificato, l'esecuzione parte dall'indice globale 0.
  - `gws` è il global work size, cioè il numero totale di work-item da eseguire.
  - `NULL` è il local work size. Non essendo specificato, la sua scelta è demandata al runtime OpenCL.
  - `0` indica che non ci sono eventi da attendere prima dell'esecuzione.
  - `NULL` è la lista degli eventi da attendere, che in questo caso è vuota.
  - `&init_evt` è un puntatore alla variabile evento in cui viene salvato l'evento associato al lancio del kernel.

3.5 Recupero dei dati

Dopo l'esecuzione del kernel, è necessario recuperare i dati dal dispositivo per renderli disponibili sull'host. Per fare questo si utilizza la funzione `clEnqueueReadBuffer`, che legge i dati da un buffer OpenCL e li copia in una regione di memoria host. Il codice seguente mostra come fare:

```
int *h_vec = malloc(sizeof(int) * n);
if (!h_vec) {
    fprintf(stderr, "allocazione fallita\n");
    exit(3);
}

cl_event wait_list[1] = { init_evt };
cl_event read_evt;

err = clEnqueueReadBuffer(
    que,            /* coda dei comandi */
    d_vec,          /* buffer da cui leggere */
    CL_TRUE,        /* chiamata bloccante per l'host */
    0, memsize,     /* offset e numero di byte da copiare */
    h_vec,          /* destinazione in memoria host */
    1, wait_list,   /* evento da attendere prima della lettura */
    &read_evt
);    /* evento associato alla lettura */

ocl_check(err, "read buffer d_vec");
```

Listing 3.5: Recupero dei dati da un buffer OpenCL

Descrizione del codice

Le varie righe possono essere descritte così:

- `int *h_vec = malloc(sizeof(int) * n)` alloca memoria sul lato host per contenere i dati letti dal buffer OpenCL. In particolare, vengono allocati `n` elementi di tipo `int`.
- `if (!h_vec)` è la verifica che l'allocazione sia andata a buon fine. In caso contrario, viene stampato un messaggio di errore e l'esecuzione del programma viene terminata.
- `cl_event wait_list[1] = { init_evt }` è un array di eventi che rappresenta la lista di comandi che devono essere completati prima di poter eseguire la lettura. In questo caso, si aspetta il completamento del kernel associato all'evento `init_evt`.
- `cl_event read_evt` è una variabile che conterrà l'evento associato all'operazione di lettura dal buffer.
- `clEnqueueReadBuffer(...)` è la funzione che accoda il comando di lettura del buffer dalla memoria del device alla memoria host. I parametri sono:
  - `que` è la command queue su cui viene accodata l'operazione.
  - `d_vec` è il buffer OpenCL da cui vengono letti i dati.

  - `CL_TRUE` indica che la chiamata è bloccante per l'host, cioè la funzione ritorna solo quando i dati sono stati completamente copiati.
  - `0` è l'offset in byte da cui iniziare la lettura.
  - `memsize` è il numero di byte da copiare.
  - `h_vec` è il puntatore alla memoria host in cui vengono copiati i dati.
  - `1` è il numero di eventi nella wait list.
  - `wait_list` è la lista di eventi che devono essere completati prima di eseguire la lettura.
  - `&read_evt` è un puntatore alla variabile in cui viene salvato l'evento associato all'operazione di lettura.

- `ocl_check(err, "read buffer d_vec")` verifica che l'operazione sia andata a buon fine. In caso di errore, esso viene segnalato e l'esecuzione viene interrotta.

Capitolo 4

Ottimizzazione e gestione dell'esecuzione in OpenCL

4.1 Mapping al posto di read

Nella versione precedente, dopo l'esecuzione del kernel, i dati prodotti sul device venivano trasferiti all'host tramite una copia esplicita. Questo richiedeva due operazioni distinte:

- allocare un vettore lato host con `malloc`;
- copiare i dati dal device con `clEnqueueReadBuffer`.

Questo approccio è semplice e immediato, ma introduce un overhead dovuto sia alla gestione separata della memoria sia alla copia esplicita dei dati.

L'idea del mapping è quella di evitare questa copia esplicita e permettere all'host di accedere direttamente al buffer OpenCL. In pratica:

- il buffer viene allocato in modo compatibile con l'accesso da host;
- l'host ottiene un puntatore al buffer tramite `clEnqueueMapBuffer`;
- una volta terminato l'utilizzo, il buffer viene rilasciato con `clEnqueueUnmapMemObject`.

In questo modo si semplifica la fase finale di recupero dei dati e, in molti casi, si riduce il costo complessivo del trasferimento.

4.2 Modifica nella creazione del buffer

La prima modifica riguarda la creazione del buffer. Nella versione originale si aveva:

```
cl_mem d_vec = clCreateBuffer(ctx, CL_MEM_WRITE_ONLY, memsize, NULL, &err);
```

Listing 4.1: Creazione del buffer senza mapping

Nella versione ottimizzata si utilizza invece:

```
cl_mem d_vec = clCreateBuffer(
  ctx,
  CL_MEM_WRITE_ONLY | CL_MEM_ALLOC_HOST_PTR,
  memsize,
  NULL,
```

```
  &err
);
```

Listing 4.2: Creazione del buffer con mapping

Il flag `CL_MEM_ALLOC_HOST_PTR` indica al runtime (la libreria OpenCL) di allocare il buffer in una regione di memoria adatta a essere mappata dall'host. Non si tratta quindi di una semplice allocazione generica sul device, ma di una scelta che prepara il buffer a una successiva fase di accesso diretto dal lato host.

4.3 Recupero dei dati con il mapping

La seconda modifica riguarda il recupero dei dati. Nella versione precedente si utilizzava `clEnqueueReadBuffer`, mentre nella nuova versione si ricorre al mapping del buffer:

```
cl_event wait_list[1] = { init_evt };
cl_event read_evt;

cl_int *h_vec = clEnqueueMapBuffer(que, d_vec, CL_TRUE,
                  CL_MAP_READ, 0, memsize, 1, wait_list, &read_evt, &err);
ocl_check(err, "read buffer d_vec");
```

Listing 4.3: Mapping del buffer

Questa funzione restituisce direttamente un puntatore, in questo caso `h_vec`, che può essere utilizzato dal codice host per leggere il contenuto del buffer. Dal punto di vista logico, non si sta più chiedendo al runtime di copiare i dati in un vettore separato, ma di rendere accessibile il contenuto del buffer stesso.

4.4 Accesso ai dati

Una volta ottenuto il puntatore, i dati possono essere usati direttamente, quindi non è più necessario effettuare una copia esplicita dei dati in una struttura allocata manualmente sull'host. Il risultato è un codice più lineare e, in generale, un percorso di trasferimento più efficiente.

Problemi di swap e pinning. Per ottenere trasferimenti efficienti tra host e device, il runtime OpenCL sfrutta normalmente operazioni di DMA (*Direct Memory Access*), che permettono di copiare i dati senza coinvolgere direttamente la CPU. Questo tipo di trasferimento è vantaggioso perché riduce il lavoro del processore, ma introduce anche alcune problematiche legate alla gestione della memoria host.

La memoria dell'host, infatti, è controllata dal sistema operativo e può essere soggetta a operazioni come swap o rilocazione delle pagine. Se durante un trasferimento DMA quella regione di memoria viene spostata o modificata dal sistema operativo, si possono verificare inconsistenze nei dati o errori nel trasferimento.

Per evitare questo problema, è necessario garantire che la memoria coinvolta rimanga stabile per tutta la durata dell'operazione. Questo si ottiene tramite il *pinning*, cioè il blocco delle pagine di memoria host in modo che non possano essere spostate. In OpenCL questo comportamento è favorito dall'uso della flag `CL_MEM_ALLOC_HOST_PTR`, che consente al runtime di predisporre una memoria compatibile con questo tipo di gestione.

Inoltre, bisogna ricordare che l'host non deve accedere alla memoria mappata prima che l'operazione sia stata completata. Questo vincolo può essere rispettato utilizzando eventi OpenCL oppure operazioni bloccanti, così da evitare race condition o accessi incoerenti ai dati.

4.5 Fase di unmapping

Dopo aver utilizzato i dati, è necessario rilasciare il mapping:

```
cl_event unmap_evt;
err = clEnqueueUnmapMemObject(que, d_vec, h_vec, 0, NULL, &unmap_evt);
ocl_check(err, "unmap buffer d_vec");
```

Listing 4.4: Unmapping del buffer

Questa operazione segnala al runtime che l'host ha terminato di accedere al buffer. Da questo momento in poi, il buffer torna a essere gestito normalmente dalla libreria OpenCL e non deve più essere usato tramite il puntatore ottenuto con il mapping.

Nel complesso, l'uso del mapping rappresenta una piccola ma importante ottimizzazione: il flusso dei dati tra device e host diventa più diretto, si elimina una copia esplicita e si sfrutta in modo più efficiente la memoria resa disponibile dal runtime.

4.6 Numeri della griglia di lancio

Generalmente il numero di work-item per blocco (local size) e il numero di blocchi (global size) sono scelti dal driver in modo da massimizzare l'occupazione del device, tenendo conto delle limitazioni hardware. Esistono però casi in cui è necessario specificare esplicitamente questi parametri, ad esempio per aumentare le prestazioni.

4.6.1 Global size ideale

La global size ideale è quella che permette di sfruttare al meglio le risorse del device, evitando sia di avere troppi work-item inattivi (sotto-occupazione) sia di avere un numero eccessivo di work-item che competono per le stesse risorse (sovraccarico). Questo si ottiene scegliendo una global size che sia un multiplo del numero di work-item per blocco, in modo da garantire che ogni blocco sia completamente occupato.

Esempio. Supponiamo di avere un device che supporta 256 work-item per blocco. Se si sceglie una global size di 1024, si avranno 4 blocchi completamente occupati, il che è ideale. Se invece si sceglie una global size di 1000, si avranno 3 blocchi completamente occupati e un quarto blocco con solo 232 work-item, il che porta a una sotto-occupazione e a prestazioni inferiori.

4.6.2 Numeri primi

Un caso particolare è quello in cui il numero di elementi da processare è un numero primo. In questo caso, se si sceglie una global size pari al numero di elementi, si avrà un solo blocco con un solo work-item, il che non sfrutta affatto la potenza del device. In questi casi è necessario scegliere una global size maggiore, magari arrotondando al multiplo più vicino di un numero di work-item per blocco che sia efficiente (ad esempio 256 o 512).

Il codice device va quindi riscritto nel seguente modo:

```
kernel void init_k(global int *vec, int nels)
{
    int i = get_global_id(0);
    if (i >= nels) return;
    vec[i] = i;
}
```

Listing 4.5: Scelta della preferred global size - codice device

Rispetto al codice precedente, passiamo al kernel anche il parametro `nels` e introduciamo un controllo dell'indice corrente per verificare che non superi il numero effettivo di elementi.

Per quanto riguarda il codice host, modifichiamo il main per calcolare la quantità `wg_mul`, che coincide con il numero di cui la global size deve essere multiplo in base all'hardware utilizzato. Il codice è il seguente:

```
size_t wg_mul;
err = clGetKernelWorkGroupInfo(init_k, d,
    CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE, sizeof(wg_mul), &wg_mul, NULL);
ocl_check(err, "preferred work-group size multiple fallito", kname);

cl_event init_evt = init(que, init_k, d_vec, nels, wg_mul);
```

Listing 4.6: Scelta della preferred global size - codice host (main)

Descrizione del codice

- `clGetKernelWorkGroupInfo(...)` è la funzione che permette di interrogare il sistema sulle capacità e i limiti di un kernel specifico quando questo viene eseguito su un determinato device. La funzione ritorna un codice di errore che viene poi controllato con la funzione `ocl_check`. I parametri sono:
  - `init_k` è il kernel per il quale si vogliono chiedere informazioni.
  - `d` è l'id del device per il quale si vogliono chiedere informazioni.
  - `CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE` è l'informazione che si vuole ottenere, ovvero il divisore ideale per la dimensione del work group.
  - `sizeof(wg_mul)` è la dimensione in byte della variabile in cui conservare l'informazione richiesta.
  - `&wg_mul` è il puntatore alla variabile in cui memorizzare l'informazione.
  - `NULL` è un puntatore opzionale a una variabile `size_t` dove OpenCL scriverebbe il numero di byte effettivamente restituiti. Di solito si passa NULL se si sa già cosa aspettarsi.
- Rispetto al codice precedente, passiamo alla funzione `init()` anche i parametri `nels` e `wg_mul`.

La funzione `init()` all'interno del codice host va invece modificata nel seguente modo:

```
cl_event init(cl_command_queue que, cl_kernel init_k,
                cl_mem vec, cl_int nels, size_t wg_mul)
{
    cl_int err;
    cl_event init_evt;

    const size_t gws[1] = { round_mul_up(nels, wg_mul) };
    printf("%d | %zu = %zu\n", nels, wg_mul, gws[0]);

    cl_uint arg_index = 0;
    err = clSetKernelArg(init_k, arg_index, sizeof(cl_mem), &vec);
    ocl_check(err, "init_k set kernel arg %u", arg_index);
    ++arg_index;
    err = clSetKernelArg(init_k, arg_index, sizeof(nels), &nels);
    ocl_check(err, "init_k set kernel arg %u", arg_index);

    err = clEnqueueNDRangeKernel(que, init_k, 1, NULL, gws,
        NULL, 0, NULL, &init_evt);

    ocl_check(err, "enqueue kernel init_k");
    return init_evt;
}
```

Listing 4.7: Scelta della preferred global size - codice host (init)

Descrizione del codice

Le modifiche apportate rispetto al codice precedente sono le seguenti:

- La global work size `gws` viene calcolata arrotondando il valore di `nels` in modo da renderlo un multiplo di `wg_mul`.
- Passiamo al kernel anche l'argomento `nels` usando la funzione `clSetKernelArg()`.

In questo modo si garantisce che ogni blocco sia completamente occupato anche se il numero di elementi è un numero primo.

4.7 Aliasing

L'aliasing è una situazione in cui due o più puntatori fanno riferimento alla stessa area di memoria. In OpenCL, l'aliasing può portare a problemi di coerenza dei dati e a prestazioni inferiori, poiché il runtime potrebbe dover inserire operazioni di sincronizzazione o di copia per garantire la correttezza del programma.

Per evitare problemi di aliasing, è importante progettare i kernel in modo che ogni work-item acceda a una porzione distinta della memoria. Inoltre, è importante pensare che il compilatore debba essere libero di fare ottimizzazioni con la sicurezza che non si verifichi l'aliasing. Un modo per farlo è utilizzare la keyword `restrict` nei puntatori per indicare al compilatore che i dati a cui si accede non sono condivisi con altri puntatori. In questo modo il compilatore può applicare ottimizzazioni più aggressive, migliorando le prestazioni del kernel.

Esempio: assegnazione per n volte di un valore a una zona di memoria. Ipotizzando di avere il seguente kernel (molto inefficiente) che assegni lo stesso valore ad un elemento di array per n volte:

```
kernel void assign_value(global float *data, float value, int n) {
    int gid = get_global_id(0);
    for (int i = 0; i < n; i++) {
        data[gid] = value;
    }
}
```

Listing 4.8: Kernel con potenziale aliasing

Si può notare che, nonostante il codice esegua la stessa operazione più volte, il compilatore non può comunque assumere che le operazioni siano uguali. Infatti, vi è la possibilità che ci sia un aliasing, ovvero che il puntatore `data` possa essere condiviso con altri puntatori. Di conseguenza, il compilatore potrebbe decidere di non ottimizzare il ciclo, eseguendo tutte le iterazioni del ciclo for.

Se invece si utilizza la keyword `restrict` per indicare che il puntatore non è condiviso, il kernel potrebbe essere riscritto in questo modo:

```
kernel void assign_value(global float *restrict data, float value, int n) {
    int gid = get_global_id(0);
    for (int i = 0; i < n; i++) {
        data[gid] = value;
    }
}
```

Listing 4.9: Kernel con `restrict` per evitare aliasing

In questo caso, il compilatore può assumere che le operazioni all'interno del ciclo for siano identiche e può quindi ottimizzare il codice, ad esempio eliminando il ciclo e inizializzando il valore di `data[gid]` una sola volta. Questo può portare a un miglioramento significativo delle prestazioni, soprattutto se n è grande.

4.8 Vettorizzazione

La *vettorizzazione* è una tecnica di ottimizzazione che consiste nel far elaborare a ciascun work-item più elementi contemporaneamente, anziché uno solo. Questo riduce il numero totale di work-item necessari per processare un dataset, abbassando l'overhead di scheduling e permettendo di sfruttare più efficacemente le risorse hardware.

In OpenCL esistono due modalità principali per applicare la vettorizzazione:

Stride scalare Ciascun work-item accede a più elementi usando variabili scalari distinte e indici calcolati esplicitamente. Il programmatore gestisce manualmente il calcolo degli indici.

Tipi vettoriali OpenCL mette a disposizione tipi di dato vettoriali nativi (`int2`, `int4`, `int8`, `int16`, ecc.) che permettono di operare su più elementi con una singola istruzione, sfruttando direttamente le unità SIMD dell'hardware.

Per illustrare e confrontare i vari approcci si userà come esempio la somma element-wise di due vettori `vec1` e `vec2` con risultato in `out`.

4.8.1 Kernel di inizializzazione

Il kernel `init_k` inizializza i due vettori di input: `vec1[i] = i` e `vec2[i] = nels - i`, in modo che la somma attesa sia `out[i] = nels` per ogni indice `i`.

```
kernel void init_k(global int * restrict vec1,
                   global int * restrict vec2, int nels)
{
    int i = get_global_id(0);
    if (i >= nels) return;
    vec1[i] = i;
    vec2[i] = nels - i;
}
```

Listing 4.10: Kernel di inizializzazione dei vettori di input

4.8.2 Caso base

La baseline di partenza è `sum_k`, dove ogni work-item elabora esattamente un elemento:

```
// indici nell'array:    0 1 2 3 4 5 6 7 ...
// indici del work-item: 0 1 2 3 4 5 6 7 ...
kernel void sum_k(global int * restrict out,
    global int const * restrict vec1,
    global int const * restrict vec2,
    int nels)
{
    int i = get_global_id(0);
    if (i >= nels) return;
    out[i] = vec1[i] + vec2[i];
}
```

Listing 4.11: Kernel di somma base: 1 work-item per elemento

La global work size è pari a `nels`. La coalescenza è ottimale: work-item adiacenti accedono a posizioni adiacenti in memoria (stride 1), consentendo al controller di aggregare tutte le richieste di un warp[1] in un'unica transazione.

4.8.3 Stride scalare contiguo

Un primo approccio consiste nel far elaborare a ciascun work-item un blocco di s elementi *consecutivi*. Con $s = 2$:

```
// indici nell'array:    0 1 2 3 4 5 6 7 ...
// indici del work-item: 0 0 1 1 2 2 3 3 ...
kernel void sum_s2_k(global int * restrict out,
    global int const * restrict vec1,
    global int const * restrict vec2,
    int nels)
{
    int wi = get_global_id(0);
```

[1]Un warp è un gruppo di work-item che vengono eseguiti simultaneamente su una unità di elaborazione.

```
    int gi0 = 2*wi;
    int gi1 = 2*wi + 1;

    if (gi0 < nels) out[gi0] = vec1[gi0] + vec2[gi0];
    if (gi1 < nels) out[gi1] = vec1[gi1] + vec2[gi1];
}
```

Listing 4.12: Stride scalare contiguo, fattore 2

Con fattore 4 (`sum_s4_k`) il work-item `wi` gestisce gli indici `4*wi`, `4*wi+1`, `4*wi+2`, `4*wi+3`. In entrambi i casi la global work size scende a $\lceil \texttt{nels}/s \rceil$.

Coalescenza. I work-item adiacenti accedono a blocchi contigui ma non sovrapposti: il work-item 0 legge gli indici 0–1, il work-item 1 legge 2–3, ecc. All'interno di un warp di W work-item, il primo accesso copre le posizioni $0, 2, 4, \ldots, 2(W-1)$: la distanza tra work-item adiacenti è s, non 1. La coalescenza è quindi peggiore rispetto al caso base.

4.8.4 Stride scalare distribuito

Una variante più efficiente è lo stride *distribuito*: ciascun work-item con indice `wi` elabora elementi separati dalla global work size (`gws`), ovvero `wi`, `wi + gws`, `wi + 2*gws`, ecc.

```
// indici nell'array:    0 1 2 3 4 5 6 7 ...
// indici del work-item: 0 1 2 3 0 1 2 3 ...
kernel void sum_s2s_k(global int * restrict out,
    global int const * restrict vec1,
    global int const * restrict vec2,
    int nels)
{
    int wi = get_global_id(0);
    int gi0 = wi;
    int gi1 = wi + get_global_size(0);

    if (gi0 < nels) out[gi0] = vec1[gi0] + vec2[gi0];
    if (gi1 < nels) out[gi1] = vec1[gi1] + vec2[gi1];
}
```

Listing 4.13: Stride scalare distribuito, fattore 2

Con fattore 4 (`sum_s4s_k`) gli indici diventano `wi`, `wi+gws`, `wi+2*gws`, `wi+3*gws`.

Coalescenza. Per il primo accesso (`gi0 = wi`), work-item adiacenti leggono posizioni adiacenti del vettore (stride 1). Lo stesso vale per ogni accesso successivo (`gi1`, `gi2`, `gi3`), che coprono blocchi contigui separati di `gws` elementi. La coalescenza è ottimale, identica al caso base, a differenza dello stride contiguo.

Confronto tra stride contiguo e distribuito

I due approcci producono lo stesso numero di accessi in memoria e la stessa riduzione della global work size, ma si comportano diversamente rispetto alla coalescenza. Nel caso distribuito, ogni accesso di un warp è a stride 1, il che permette al controller di servire ciascuna fase di lettura

con un numero minimo di transazioni. Per questo motivo lo stride distribuito è generalmente preferibile.

4.8.5 Separazione delle fasi di load e store

Un ulteriore raffinamento dello stride distribuito a fattore 4 è `sum_s4sx_k`, che separa esplicitamente la fase di caricamento da quella di calcolo e scrittura:

```
kernel void sum_s4sx_k(global int * restrict out,
    global int const * restrict vec1,
    global int const * restrict vec2,
    int nels)
{
    const int gws = get_global_size(0);
    const int wi  = get_global_id(0);
    const int gi0 = wi;
    const int gi1 = wi +   gws;
    const int gi2 = wi + 2*gws;
    const int gi3 = wi + 3*gws;

    const int v10 = gi0 < nels ? vec1[gi0] : 0;
    const int v20 = gi0 < nels ? vec2[gi0] : 0;
    const int v11 = gi1 < nels ? vec1[gi1] : 0;
    const int v21 = gi1 < nels ? vec2[gi1] : 0;
    const int v12 = gi2 < nels ? vec1[gi2] : 0;
    const int v22 = gi2 < nels ? vec2[gi2] : 0;
    const int v13 = gi3 < nels ? vec1[gi3] : 0;
    const int v23 = gi3 < nels ? vec2[gi3] : 0;

    if (gi0 < nels) out[gi0] = v10 + v20;
    if (gi1 < nels) out[gi1] = v11 + v21;
    if (gi2 < nels) out[gi2] = v12 + v22;
    if (gi3 < nels) out[gi3] = v13 + v23;
}
```

Listing 4.14: Stride distribuito, fattore 4, con load separati

Emettere tutti i caricamenti prima che i risultati siano necessari permette all'hardware di avviare più richieste di memoria in parallelo, nascondendo la latenza (*latency hiding*). Dichiarare le variabili come `const` fornisce inoltre al compilatore la garanzia che i valori non cambieranno, facilitando il riordinamento delle istruzioni e ottimizzazioni ulteriori.

4.8.6 Tipi vettoriali

OpenCL fornisce tipi di dato vettoriali nativi — `int2`, `int4`, `int8`, `int16` — che raggruppano rispettivamente 2, 4, 8 e 16 elementi dello stesso tipo. Gli operatori aritmetici su questi tipi vengono applicati component-wise con una singola istruzione SIMD. Il kernel di somma con `int4` è:

```
// indici nell'array:    0 1 2 3 4 5 6 7 ...
// indici del work-item: 0 0 0 0 1 1 1 1 ...
kernel void sum_v4_k(global int4 * restrict out,
```

```
    global int4 const * restrict vec1,
    global int4 const * restrict vec2,
    int quarts)
{
    int wi = get_global_id(0);
    if (wi >= quarts) return;
    out[wi] = vec1[wi] + vec2[wi];
}
```

Listing 4.15: Kernel di somma con tipo vettoriale `int4`

Il parametro `quarts` $= \lceil$`nels`$/4\rceil$ è il numero di gruppi di 4 elementi. Il codice è identico nella struttura al caso base, ma ogni work-item opera su un `int4`: l'addizione esegue 4 somme a livello hardware con una sola istruzione. La stessa struttura vale per `sum_v2_k`, `sum_v8_k` e `sum_v16_k`, che usano rispettivamente `int2`, `int8` e `int16`.

Coalescenza. Il pattern di accesso è equivalente allo stride contiguo: il work-item 0 carica gli elementi [0–3], il work-item 1 carica [4–7], ecc. I blocchi acceduti da work-item adiacenti sono contigui, garantendo una buona coalescenza.

Vantaggi rispetto allo stride scalare. Sebbene il pattern di accesso sia simile allo stride scalare contiguo, i tipi vettoriali presentano vantaggi significativi:

- Il compilatore genera istruzioni SIMD native senza dover dipendere dall'auto-vettorizzazione.
- Il codice risulta più compatto e leggibile rispetto alle versioni scalari con stride.
- L'espressione `vec1[wi] + vec2[wi]` comunica esplicitamente l'intenzione vettoriale, riducendo i margini di errore.

4.8.7 Sliding window

Il kernel `sum_sliding_k` generalizza l'approccio a stride distribuito usando un ciclo `while` che avanza di `gws` ad ogni iterazione, anziché un numero fisso di accessi:

```
// indici nell'array:    0 1 2 3 4 5 6 7 ...
// indici del work-item: 0 1 2 3 0 1 2 3 ...
kernel void sum_sliding_k(global int * restrict out,
    global int const * restrict vec1,
    global int const * restrict vec2,
    int nels)
{
    int wi = get_global_id(0);
    while (wi < nels) {
        out[wi] = vec1[wi] + vec2[wi];
        wi += get_global_size(0);
    }
}
```

Listing 4.16: Kernel di somma con sliding window

La coalescenza è ottimale per le stesse ragioni dello stride distribuito. Il vantaggio principale rispetto alle versioni a stride fisso è la flessibilità: la global work size può essere scelta indipendentemente da `nels` — ad esempio un multiplo della preferred work-group size — e il ciclo si occupa automaticamente degli elementi rimanenti. Questo è particolarmente utile quando `nels` non è noto al momento della compilazione o varia a runtime. Uno degli svantaggi è che il ciclo introduce un overhead di controllo e il compilatore non può applicare ottimizzazioni al caricamento in memoria centrale come nel caso stride scalare distribuito 4.13 o con tipi vettoriali 4.15.

Confronto degli approcci

Kernel	Elem. per work-item	Coalescenza	Note
`sum_k`	1	Ottimale	Caso base
`sum_s2_k`	2 contigui	Degradato	Stride tra WI $= s$
`sum_s4_k`	4 contigui	Degradato	Stride tra WI $= s$
`sum_s2s_k`	2 distribuiti	Ottimale	Stride tra WI $= 1$
`sum_s4s_k`	4 distribuiti	Ottimale	Stride tra WI $= 1$
`sum_s4sx_k`	4 distribuiti	Ottimale	Load separati dagli store
`sum_v2_k`	2 (int2)	Buono	SIMD nativo
`sum_v4_k`	4 (int4)	Buono	SIMD nativo
`sum_v8_k`	8 (int8)	Buono	SIMD nativo
`sum_v16_k`	16 (int16)	Buono	SIMD nativo
`sum_sliding_k`	variabile	Ottimale	GWS indipendente da `nels`

Tabella 4.1: Confronto tra gli approcci di vettorizzazione per il kernel di somma.

4.9 Matrici in OpenCL

4.9.1 Rappresentazione in memoria

In C, le matrici seguono il layout *row-major*: l'indice di colonna è quello che progredisce più velocemente in memoria. Per una matrice `mat[3][2]`, il layout in memoria è:

```
mat[0][0] mat[0][1] mat[1][0] mat[1][1] mat[2][0] mat[2][1]
```

Esistono tre modi per allocare dinamicamente una matrice in C:

Doppio puntatore con righe sparse Le righe vengono allocate separatamente. L'accesso è identico agli array statici (`mat[r][c]`), ma le righe si trovano in posizioni sparse in memoria e serve spazio aggiuntivo per i puntatori alle righe.

```
int **mat;
mat = malloc(R * sizeof(int *));
for (int r = 0; r < R; ++r)
    mat[r] = malloc(C * sizeof(int));
```

Listing 4.17: Matrice con righe sparse

Doppio puntatore con blocco unico L'intera matrice è allocata in un unico blocco contiguo, ma si mantiene comunque il doppio livello di indirezione. Ogni accesso richiede due letture in memoria: prima si recupera il puntatore alla riga, poi si legge l'elemento.

```
int **mat;
mat = malloc(R * sizeof(int *));
mat[0] = malloc(R * C * sizeof(int));
for (int r = 1; r < R; ++r)
    mat[r] = mat[r-1] + C;
```

Listing 4.18: Matrice in blocco unico con doppio puntatore

Array linearizzato La matrice è un puntatore semplice allocato in un unico blocco. L'accesso richiede un unico accesso in memoria tramite la formula `mat[r*C + c]`, ma richiede di conoscere a priori il numero di colonne.

```
int *mat;
mat = malloc(R * C * sizeof(int));
mat[r * C + c] = /* valore */;
```

Listing 4.19: Matrice linearizzata

Nella GPGPU si lavora esclusivamente con array linearizzati per tre motivi: un solo accesso in memoria per elemento, nessun doppio livello di indirezione, e l'impossibilità di associare tra loro due allocazioni separate sulla GPU (i puntatori host non sono validi sul device).

4.9.2 Griglia di lancio 2D

Per operare su una matrice, è naturale usare una griglia di lancio bidimensionale: ogni work-item riceve un indice di riga e uno di colonna.

Fino ad ora la local work size (LWS) era sempre `NULL`, delegando la scelta al driver. In questo caso la specifichiamo esplicitamente: la LWS è un quadrato di lato `lws_cli`, quindi ogni work-group contiene `lws_cli`×`lws_cli` work-item. Specificare la LWS esplicitamente impone che la GWS sia un suo multiplo esatto lungo ogni dimensione; per questo si usa `round_mul_up` anche sulla dimensione delle colonne.

```
kernel void init_k(global int *mat, int rows, int cols)
{
    int r = get_global_id(0);
    int c = get_global_id(1);

    if (r < rows && c < cols) mat[r * cols + c] = r - c;
}
```

Listing 4.20: Kernel di inizializzazione 2D (prima versione)

```
const size_t lws[2] = { lws_cli, lws_cli };
const size_t gws[2] = { round_mul_up(nrows, lws[0]),
                        round_mul_up(ncols, lws[1]) };

err = clEnqueueNDRangeKernel(que, init_k, 2, NULL, gws, lws,
                             0, NULL, &init_evt);
```

Listing 4.21: Lancio 2D lato host

Il parametro `2` in `clEnqueueNDRangeKernel` specifica che la griglia è bidimensionale. Il controllo `if (r < rows && c < cols)` è necessario perché `round_mul_up` può produrre una GWS leggermente superiore alle dimensioni effettive della matrice.

4.9.3 Coalescenza in 2D e orientamento del warp

La versione precedente ha prestazioni non ottimali a causa della coalescenza degli accessi. Nell'hardware GPU i work-item vengono eseguiti in gruppi rettangolari (wavefront o warp). Se il work group è un quadrato di 16×16 work-item e la warp size è 32, il wavefront potrebbe essere organizzato come un rettangolo 4×8 o 8×4: la forma dipende da quale asse avanza più velocemente.

L'asse `0` (`get_global_id(0)`) è quello che avanza più velocemente all'interno di un wavefront. Per ottenere accessi coalescenti, questo asse deve corrispondere all'indice di colonna: in questo modo work-item con indici progressivi accedono a posizioni contigue in memoria, e il controller può servire tutte le richieste del wavefront con una singola transazione.

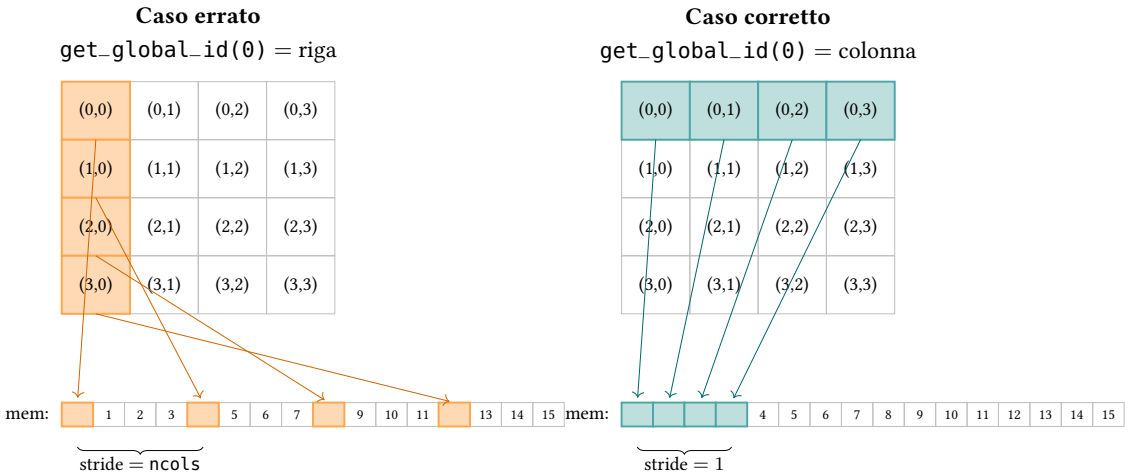

Figura 4.1: Effetto dell'orientamento del wavefront sulla coalescenza in 2D, con work-group 4×4 e wavefront di 4 work-item. A sinistra, con `get_global_id(0)` = riga, il wavefront è verticale (colonna) e gli accessi in memoria sono distanziati di `ncols` posizioni (arancione). A destra, con `get_global_id(0)` = colonna, il wavefront è orizzontale (riga) e gli accessi sono contigui (turchese).

La correzione richiede di scambiare gli indici nel kernel:

```
kernel void init_k(global int *mat, int rows, int cols)
{
    int r = get_global_id(1); /* asse 1: avanza lentamente */
    int c = get_global_id(0); /* asse 0: avanza velocemente */

    if (r < rows && c < cols) mat[r * cols + c] = r - c;
}
```

Listing 4.22: Kernel di inizializzazione 2D (versione con coalescenza corretta)

Lato host è sufficiente scambiare `nrows` e `ncols` nella definizione della GWS:

```
const size_t gws[2] = { round_mul_up(ncols, lws[0]),
```

```
round_mul_up(nrows, lws[1]) };
```

Listing 4.23: GWS corretta per coalescenza ottimale

4.9.4 Padding (pitch)

La coalescenza richiede non solo che i dati siano contigui, ma anche che siano *allineati*: ogni riga della matrice dovrebbe iniziare a un indirizzo multiplo del base alignment dell'hardware.

Se il numero di colonne non è un multiplo del base alignment, le righe successive alla prima iniziano a indirizzi non ottimali, degradando la coalescenza. La soluzione è inserire del *padding* a fine riga, allargando la matrice fino al multiplo di colonne più vicino. Questo valore si chiama *pitch*.

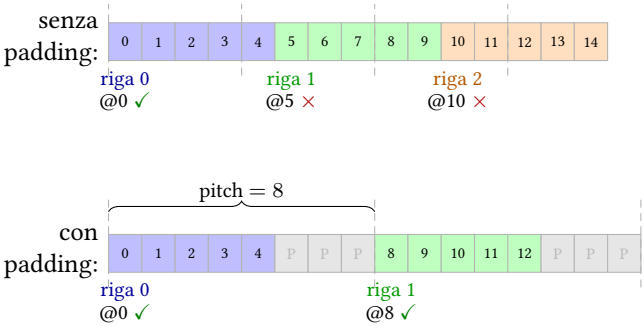

Figura 4.2: Effetto del padding sulla coalescenza. Senza padding (`ncols`= 5, allineamento= 4 elementi), le righe 1 e 2 iniziano ad indirizzi non multipli dell'allineamento (×). Con padding (`pitch`= 8), ogni riga inizia ad un indirizzo allineato (✓). Le celle P rappresentano lo spazio di padding aggiunto a fine riga.

```
const size_t base_alignment_byte = 256;
const size_t base_alignment_el   = base_alignment_byte / sizeof(cl_int);

size_t padded_pitch_el = round_mul_up(ncols, base_alignment_el);
const size_t memsize   = padded_pitch_el * nrows * sizeof(cl_int);
```

Listing 4.24: Calcolo del pitch lato host

Il kernel riceve `pitch_el` come parametro aggiuntivo e lo usa al posto di `cols` per calcolare l'offset di riga:

```
kernel void init_padding_k(global int *mat, int rows, int cols, int pitch_el)
{
    int r = get_global_id(1);
    int c = get_global_id(0);

    if (r < rows && c < cols) mat[r * pitch_el + c] = r - c;
}
```

Listing 4.25: Kernel con padding (pitch)

Nella funzione di verifica lato host, `pitch_el` sostituisce `ncols` nel calcolo dell'indice di riga:

```
void verify(const cl_int *mat, int nrows, int ncols, int pitch_el) {
    for (int r = 0; r < nrows; ++r)
```

```
        for (int c = 0; c < ncols; ++c) {
            int expected = r - c;
            int computed = mat[r * pitch_el + c];
            if (expected != computed) { /* errore */ }
        }
}
```

Listing 4.26: Verifica con pitch

Capitolo 5

Local memory e sincronizzazione

In questo capitolo si introduce la local memory di OpenCL e i meccanismi di sincronizzazione necessari per usarla correttamente. I concetti vengono presentati in modo progressivo attraverso quattro versioni del kernel *vecsmooth*: scalare, vettoriale, con local memory e vettoriale con local memory.

5.1 Il problema: smoothing di un vettore

Il problema consiste nel calcolare, per ogni elemento di un vettore di interi, la media tra l'elemento stesso e i suoi vicini immediati:

$$\texttt{out}[i] = \frac{\texttt{in}[i-1] + \texttt{in}[i] + \texttt{in}[i+1]}{3}$$

con casi limite agli estremi: il primo elemento usa solo `in`[0] e `in`[1], l'ultimo usa `in`[nels-2] e `in`[nels-1].

5.2 Versione scalare

La versione base assegna un work-item per elemento. Usa un accumulatore `acc` e un divisore `div` aggiornati solo se l'elemento adiacente esiste, tramite due `if` indipendenti.

```
kernel void smooth_k(global int * restrict out,
                     global const int * restrict in, int nels)
{
    int i = get_global_id(0);
    if (i >= nels) return;

    int acc = in[i];
    int div = 1;

    if (i > 0) {
        acc += in[i-1];
        ++div;
    }
    if (i < nels - 1) {
        acc += in[i+1];
```

```
        ++div;
    }

    out[i] = acc/div;
}
```

Listing 5.1: Kernel smooth scalare

Lato host la funzione `smooth()` calcola la GWS come multiplo della LWS tramite `round_mul_up`:

```
cl_event smooth(cl_command_queue que, cl_kernel smooth_k,
    cl_mem out, cl_mem in, cl_int nels,
    size_t lws_cli, cl_event init_evt)
{
    const size_t lws[1] = { lws_cli };
    const size_t gws[1] = { round_mul_up(nels, lws[0]) };

    cl_uint arg_index = 0;
    clSetKernelArg(smooth_k, arg_index++, sizeof(cl_mem), &out);
    clSetKernelArg(smooth_k, arg_index++, sizeof(cl_mem), &in);
    clSetKernelArg(smooth_k, arg_index++, sizeof(nels),   &nels);

    cl_event smooth_evt;
    clEnqueueNDRangeKernel(que, smooth_k, 1, NULL, gws, lws,
                           1, &init_evt, &smooth_evt);
    return smooth_evt;
}
```

Listing 5.2: Funzione host per il lancio del kernel smooth

I due `if` indipendenti per i bordi non sono una scelta arbitraria: la sezione seguente ne spiega il costo rispetto a un `if/else`.

La versione scalare rilegge gli stessi dati più volte: l'elemento `in[i]` viene letto dal work-item i, ma anche dai work-item $i-1$ e $i+1$ come vicino. Le versioni successive riducono questa ridondanza.

5.3 Costo dei branch: `if` indipendenti vs `if/else`

L'hardware GPU gestisce la divergenza di flusso tramite *predication*: quando i work-item di uno stesso warp seguono percorsi diversi, il hardware esegue tutti i percorsi mascherando selettivamente i work-item che non devono contribuire al risultato.
Il costo dipende dalla struttura del condizionale:

Due `if` indipendenti Ogni condizionale viene valutato separatamente. I work-item che non soddisfano la condizione eseguono comunque le istruzioni del blocco, ma il risultato viene scartato. I due condizionali non si influenzano a vicenda: si paga solo il costo del blocco più costoso tra i due `if`.

`if/else` Tutti i work-item del wavefront eseguono *entrambi* i rami: prima il blocco `if` (con i work-item del ramo `else` mascherati), poi il blocco `else` (con quelli del ramo `if` mascherati). Il costo totale è la somma dei due rami.

Ne consegue che due `if` indipendenti che coprono casi distinti sono preferibili a un `if`/`else`: nella forma con `else` si paga sempre il costo di entrambi i rami.

5.4 Versione vettoriale

Nella versione vettoriale ogni work-item opera su una quartina `int4`. Gli elementi interni della quartina ricevono i loro vicini direttamente dagli altri componenti tramite swizzling, evitando letture ridondanti. Solo gli elementi agli estremi della quartina (`.x` e `.w`) devono leggere un elemento dalla quartina adiacente in global memory. Quella lettura usa l'aritmetica dei puntatori per accedere al singolo `int` senza caricare l'intera quartina.

```
kernel void smooth_v4_k(global int4 * restrict out,
                        global const int4 * restrict in, int nquarts)
{
    int i = get_global_id(0);
    if (i >= nquarts) return;

    const int4 val = in[i]; /* .x .y .z .w */
    /* vicini interni ottenuti per swizzling */
    int4 acc = val + (int4)(0, val.xyz) + (int4)(val.yzw, 0);
    int4 div = (int4)(2, 3, 3, 2);

    if (i > 0) {
        /* ultimo elemento della quartina precedente */
        acc.x += *((global const int *)(in + i) - 1);
        div.x += 1;
    }
    if (i < nquarts - 1) {
        /* primo elemento della quartina successiva */
        acc.w += *((global const int *)(in + i + 1));
        div.w += 1;
    }

    out[i] = (int4)acc/div;
}
```

Listing 5.3: Kernel smooth vettoriale con `int4`

Lato host le differenze rispetto alla versione scalare sono: il numero di quartine `nels/4` viene passato al kernel al posto di `nels` (con il vincolo che `nels` sia multiplo di 4), e il calcolo della banda passante tiene conto che ogni work-item legge 4 elementi propri più 2 di confine e ne scrive 4:

```
/* nels deve essere multiplo di 4 */
cl_event smooth_evt = smooth(que, smooth_k, d_out, d_in,
                             nels/4, lws, init_evt);

/* lettura: 4 propri + 2 di confine; scrittura: 4 */
printf("smooth: %.4gGB/s\n",
       (1 + 6.0/4) * memsize / (double)smooth_ns);
```

Listing 5.4: Differenze host per la versione vettoriale

Questa versione accede comunque alla global memory per ogni elemento di confine. La sezione seguente introduce la local memory, che permette di condividere dati tra work-item dello stesso work-group senza tornare alla global memory.

5.5 La local memory

La local memory (chiamata *shared memory* in CUDA) è una memoria on-chip allocata per ogni work-group al momento del lancio del kernel. A differenza della global memory, che persiste per l'intera durata del programma, la local memory è temporanea: viene allocata quando il work-group inizia l'esecuzione e viene liberata quando termina.
Le proprietà principali sono:

- È visibile solo ai work-item dello stesso work-group; work-group diversi hanno aree di local memory distinte e non possono comunicare tra loro tramite essa.
- Ha latenza molto inferiore alla global memory, poiché si trova fisicamente sulla stessa compute unit che esegue il work-group.
- La sua dimensione è limitata e influenza quanti work-group possono essere attivi contemporaneamente sulla stessa compute unit.

L'utilità principale è che i work-item dello stesso work-group girano sulla stessa compute unit: possono quindi condividere dati tramite la local memory senza passare dalla global memory, che ha latenza elevata.

Nel codice device si dichiara con la keyword `__local` (o `local`):

```
kernel void esempio_k(local int * restrict cache)
{
    /* cache: local memory, condivisa tra tutti i WI del work-group */
}
```

Listing 5.5: Dichiarazione di un parametro in local memory

5.6 Indici locali: `get_local_id()` e `get_local_size()`

Oltre all'indice globale, ogni work-item ha un indice *locale* che lo identifica all'interno del proprio work-group:

`get_global_id(0)` Indice del work-item nella griglia globale, da `0` a `GWS - 1`.

`get_local_id(0)` Indice del work-item all'interno del suo work-group, da `0` a `LWS - 1`.

`get_local_size(0)` Dimensione del work-group (= LWS).

Esempio. Con GWS $= 8$ e LWS $= 4$ si hanno due work-group:

- WG 0: `gi` $= 0, 1, 2, 3$ `li` $= 0, 1, 2, 3$
- WG 1: `gi` $= 4, 5, 6, 7$ `li` $= 0, 1, 2, 3$

L'indice locale è fondamentale per posizionare i dati nella cache condivisa: il work-item con `li`$= k$ scrive il proprio valore in `cache[k+1]` e può leggere quello del vicino in `cache[k]` e `cache[k+2]` (con l'offset di 1 della cache lws+2 descritta nella sezione seguente).

5.7 Trade-off nella dimensione della cache

Per il kernel smooth, ogni work-item ha bisogno del proprio elemento e dei due elementi adiacenti. Esistono due scelte per la dimensione della cache:

Cache di lws+2 elementi Si riserva uno slot prima del primo elemento e uno dopo l'ultimo del work-group. Il vantaggio è che il kernel legge sempre dalla cache con un accesso uniforme; lo svantaggio è un'inizializzazione più complessa, in cui il primo e l'ultimo work-item devono caricare elementi aggiuntivi dalla global memory.

Cache di lws elementi Ogni work-item scrive solo il proprio dato. L'inizializzazione è semplice, ma i work-item ai bordi del work-group devono ancora accedere alla global memory per il vicino esterno.

Nelle implementazioni che seguono si adotta la prima alternativa (lws+2): il kernel legge sempre dalla cache, rendendo il comportamento uniforme per tutti i work-item.

5.8 Race condition in local memory

I work-item dello stesso work-group sono garantiti in esecuzione contemporanea sulla stessa compute unit, ma non necessariamente sulla stessa istruzione nello stesso ciclo di clock. L'hardware li raggruppa in wavefront di dimensione fissa — tipicamente 32 o 64 work-item — e work-item appartenenti a wavefront diversi possono trovarsi in punti diversi del programma.

Se un work-group ha LWS $>$ warp_size, esso è composto da più wavefront. In questo caso è possibile una *race condition*: un work-item di un wavefront potrebbe leggere dalla local memory un valore che un work-item di un altro wavefront non ha ancora scritto.

Esempio. Con LWS$= 64$ e warp_size$= 32$, un work-group è composto da 2 wavefront. Il wavefront 0 (work-item 0–31) potrebbe avanzare all'istruzione di lettura dalla cache prima che il wavefront 1 (work-item 32–63) abbia completato le scritture. Il valore letto sarebbe indeterminato.

La soluzione è inserire una barriera di sincronizzazione tra la fase di scrittura e quella di lettura della cache.

5.9 Barriere

```
barrier(CLK_LOCAL_MEM_FENCE);
```

Listing 5.6: Barriera di sincronizzazione sulla local memory

La semantica è la seguente: tutte le scritture in local memory effettuate *prima* della barriera da qualsiasi work-item del work-group sono garantite visibili a *tutti* i work-item dello stesso work-group *dopo* la barriera. Nessun work-item può superare la barriera finché tutti gli altri non la raggiungono.

La barriera ha due vincoli importanti:

1. **Solo intra-work-group.** Non esiste una barriera a livello di griglia: work-group diversi non possono sincronizzarsi tra loro e non è garantito che siano nemmeno in esecuzione contemporaneamente.

2. **Tutti i work-item devono raggiungerla.** Se alcuni work-item escono dal kernel prima della barriera (ad esempio con un `return` anticipato), gli altri rimarranno in attesa indefinita. Il pattern corretto è spostare il controllo `if (gi >= nels) return;` *dopo* la barriera.

5.10 Allocazione di local memory dall'host

La local memory non viene allocata nel codice device: è l'host a specificarne la dimensione tramite `clSetKernelArg`, passando `NULL` come valore:

```
err = clSetKernelArg(smooth_k, arg_index,
                     (lws_cli + 2) * sizeof(cl_int), NULL);
```

Listing 5.7: Allocazione di local memory dall'host

Il `NULL` indica al runtime di allocare il numero di byte specificato come local memory per ciascun work-group, senza inizializzarli. La dimensione della cache influenza l'occupazione della compute unit: più grande è la cache, minore è il numero di work-group che possono essere attivi contemporaneamente sulla stessa CU, riducendo il margine per nascondere la latenza delle letture dalla global memory.

5.11 Versione con local memory

In questa versione ogni work-item carica il proprio elemento in una cache condivisa di `lws+2` elementi. Il primo work-item del work-group carica anche il predecessore del work-group, l'ultimo carica anche il successore. Una barriera garantisce che la cache sia completamente inizializzata prima che qualsiasi work-item la legga.

Il `return` anticipato per gli indici fuori range viene spostato *dopo* la barriera, in modo che tutti i work-item del work-group la raggiungano.

```
kernel void smooth_lmem_k(global int * restrict out,
                          global const int * restrict in,
                          int nels,
                          local int * restrict cache)
{
    const int lws = get_local_size(0);
    int gi = get_global_id(0);
    int li = get_local_id(0);

    int val, acc;
    int div = 1;

    if (gi < nels) {
        int val = in[gi];
        cache[li + 1] = val; /* elemento centrale */
```

```
        /* il primo WI del WG carica il predecessore del WG */
        if (li == 0 && gi != 0)
            cache[0] = in[gi - 1];
        /* l'ultimo WI del WG carica il successore del WG */
        if (li == lws - 1 && gi < nels - 1)
            cache[li + 2] = in[gi + 1];

        acc = val;
    }

    barrier(CLK_LOCAL_MEM_FENCE); /* tutte le scritture in cache completate */

    if (gi >= nels) return; /* return DOPO la barriera */

    if (gi > 0) {
        acc += cache[li];     /* predecessore dalla cache */
        ++div;
    }
    if (gi < nels - 1) {
        acc += cache[li + 2]; /* successore dalla cache */
        ++div;
    }

    out[gi] = acc/div;
}
```

Listing 5.8: Kernel smooth con local memory

Lato host l'unica differenza rispetto alla versione scalare è l'argomento della cache, allocato con `NULL`:

```
clSetKernelArg(smooth_k, arg_index++,
               (lws_cli + 2) * sizeof(cl_int), NULL);
```

Listing 5.9: Allocazione della cache lato host (versione lmem)

5.12 Versione vettoriale con local memory

L'ultima versione combina vettorizzazione e local memory. Poiché ogni work-item opera su una quartina, gli unici elementi che possono essere utili ai work-item vicini sono il primo (`.x`) e l'ultimo (`.w`) componente di ciascuna quartina. Considerando che non vi è alcun elemento che possa fungere sia da precedente che da successivo, si usano due cache separate di `lws` elementi ciascuna:

- `cache_prec[li]`: contiene l'ultimo elemento (`.w`) della quartina del work-item `li-1`, ovvero il predecessore della quartina del work-item `li`.
- `cache_succ[li]`: contiene il primo elemento (`.x`) della quartina del work-item `li+1`, ovvero il successore della quartina del work-item `li`.

Il primo work-item del work-group non ha un predecessore in `cache_prec` e lo carica dalla global memory; l'ultimo non ha un successore in `cache_succ` e lo carica anch'esso dalla global memory.

```
kernel void smooth_v4_lmem_k(global int4 * restrict out,
                             global const int4 * restrict in,
                             int nquarts,
                             local int * restrict cache_prec,
                             local int * restrict cache_succ)
{
    int gi = get_global_id(0);
    int li = get_local_id(0);
    const int lws = get_local_size(0);

    const int4 val = gi < nquarts ? in[gi] : (int4)(0);

    /* ogni WI fornisce il suo .w al successivo (cache_prec)
       e il suo .x al precedente (cache_succ) */
    if (li + 1 < lws) cache_prec[li + 1] = val.w;
    if (li > 0)       cache_succ[li - 1] = val.x;

    /* il primo WI del WG legge il predecessore dalla global memory */
    if (gi > 0 && li == 0)
        cache_prec[0] = *((global const int *)(in + gi) - 1);
    /* l'ultimo WI del WG legge il successore dalla global memory */
    if (gi < nquarts - 1 && li == lws - 1)
        cache_succ[li] = *((global const int *)(in + gi + 1));

    barrier(CLK_LOCAL_MEM_FENCE);

    int4 acc = val + (int4)(0, val.xyz) + (int4)(val.yzw, 0);
    int4 div = (int4)(2, 3, 3, 2);

    if (gi > 0) {
        acc.x += cache_prec[li];
        div.x += 1;
    }
    if (gi < nquarts - 1) {
        acc.w += cache_succ[li];
        div.w += 1;
    }

    if (gi < nquarts) out[gi] = acc/div;
}
```

Listing 5.10: Kernel smooth vettoriale con local memory

Lato host si allocano due cache di `lws_cli` elementi ciascuna — senza il `+2` della versione scalare — poiché le cache contengono esattamente un elemento per work-item. Come nella versione vettoriale base, si passa `nels/4` al kernel:

```
clSetKernelArg(smooth_k, arg_index++, lws_cli * sizeof(cl_int), NULL);
```

```
clSetKernelArg(smooth_k, arg_index++, lws_cli * sizeof(cl_int), NULL);

/* nels deve essere multiplo di 4 */
cl_event smooth_evt = smooth(que, smooth_k, d_out, d_in,
                             nels/4, lws, init_evt);
```

Listing 5.11: Allocazione delle due cache lato host (versione v4+lmem)

5.13 Trasposizione di una matrice

La trasposizione di una matrice $R \times C$ produce una matrice $C \times R$ in cui l'elemento in posizione (r, c) nella matrice originale compare in posizione (c, r) nella trasposta. L'implementazione più naturale (imbarazzantemente parallela) assegna un work-item per elemento della trasposta, usando `get_global_id(0)` per la colonna (asse rapido) e `get_global_id(1)` per la riga.

```
/* t_rows, t_cols: righe e colonne della trasposta */
kernel void transpose_k(global int * restrict trasposta,
                        global const int * restrict originale,
                        int t_rows, int t_cols)
{
    int t_c = get_global_id(0); /* colonna della trasposta */
    int t_r = get_global_id(1); /* riga della trasposta */

    if (t_r < t_rows && t_c < t_cols)
        trasposta[t_r * t_cols + t_c] = originale[t_c * t_rows + t_r];
}
```

Listing 5.12: Kernel naive di trasposizione

La coalescenza degli accessi è asimmetrica:

Scritture L'indice lineare `t_r * t_cols + t_c` varia di 1 al variare di `t_c` (asse 0, il più rapido): le scritture su `trasposta` sono coalescenti.

Letture L'indice lineare `t_c * t_rows + t_r` varia di `t_rows` al variare di `t_c`: le letture da `originale` hanno stride pari al numero di righe dell'originale e non sono coalescenti.

Invertendo gli assi — `get_global_id(0)` per la riga, `get_global_id(1)` per la colonna — si ottiene la situazione opposta: letture coalescenti ma scritture non coalescenti. In entrambe le varianti uno dei due accessi ha stride.

Il benchmark misura la banda passante come $2 \times$ `memsize`: si legge l'intera matrice originale e si scrive l'intera trasposta.

5.14 Letture coalescenti vs scritture coalescenti

Quando non è possibile avere entrambi gli accessi coalescenti, conviene privilegiare le *letture*. Le scritture in memoria non possono iniziare finché le letture necessarie non sono terminate: il tempo di lettura è sul percorso critico, quello di scrittura no. Ottimizzare le letture riduce quindi direttamente la latenza complessiva del kernel.

Tuttavia, anche nel caso ottimale con letture coalescenti, il kernel naive lascia le scritture con stride. La local memory consente di eliminare lo stride su *entrambi* gli accessi.

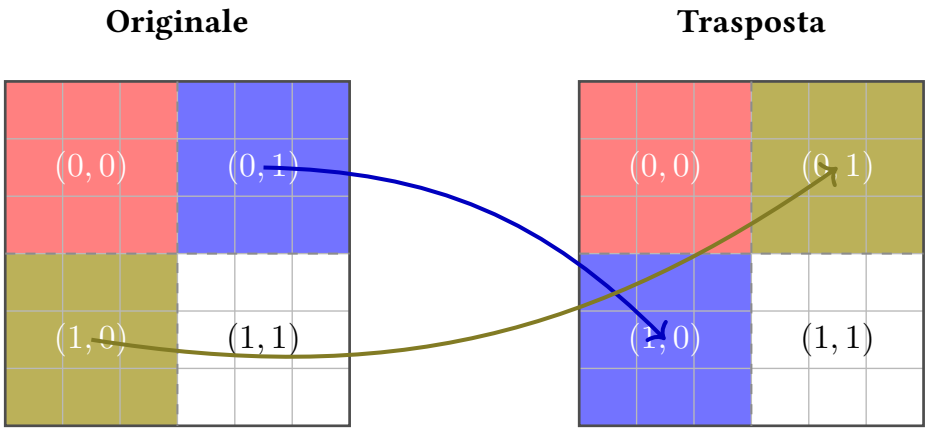

Figura 5.1: Trasposizione di una matrice 6×6 con blocchi 3×3 (`lws`$= 3$, quattro work-group). Le coordinate $(g_r,\ g_c)$ identificano il blocco nella griglia di work-group. I blocchi diagonali rimangono in posizione; i blocchi fuori diagonale si scambiano. Ciascun blocco corrisponde al tile elaborato da un work-group.

5.15 Trasposizione con tile in local memory

L'idea è dividere la matrice in *tile* quadrati di `lws`$\times$`lws` elementi, uno per work-group. Ogni work-group esegue tre fasi:

1. Carica il proprio tile dall'originale nella local memory con *letture coalescenti*.
2. Sincronizza con una barriera.
3. Scrive il tile nella posizione trasposta della matrice risultato con *scritture coalescenti*, leggendo dalla local memory con gli indici locali scambiati.

Fase 1 (caricamento). Il work-item con indici locali $(l_c,\ l_r)$ nel work-group $(g_c,\ g_r)$ legge dall'originale la posizione globale $(\text{orig_r}, \text{orig_c}) = (g_r \cdot \texttt{lws} + l_r,\ \ g_c \cdot \texttt{lws} + l_c)$. Al variare di l_c (asse 0), l'indice lineare $\text{orig_r} \cdot C + \text{orig_c}$ varia di 1: le letture sono coalescenti. Il valore viene scritto in `tile[`$l_r \cdot \texttt{lws} + l_c$`]`.

Fase 2 (scrittura). Il work-group che ha letto il blocco (g_r, g_c) dell'originale scrive nel blocco (g_c, g_r) della trasposta (coordinate di work-group scambiate). Ogni work-item $(l_c,\ l_r)$ legge `tile[`$l_c \cdot \texttt{lws} + l_r$`]` (indici locali scambiati rispetto alla fase 1) e scrive in posizione $(\text{trans_r}, \text{trans_c}) = (g_c \cdot \texttt{lws} + l_r,\ \ g_r \cdot \texttt{lws} + l_c)$. Al variare di l_c (asse 0), l'indice lineare $\text{trans_r} \cdot R + \text{trans_c}$ varia di 1: le scritture sono coalescenti.

In sintesi, in fase 2 ogni work-item scrive il valore che, in fase 1, è stato letto dal work-item con indici locali scambiati (l_r, l_c). La dimensione del tile è un trade-off: più è grande, più si riduce il numero di blocchi fuori diagonale (quelli con accessi non coalescenti) e più si aumenta la banda passante; ma più è grande, meno work-group possono essere attivi contemporaneamente sulla stessa compute unit, riducendo l'occupazione e la capacità di nascondere la latenza delle letture dalla global memory. Tipicamente si sceglie `lws` pari alla dimensione del warp/wavefront (32 o 64).

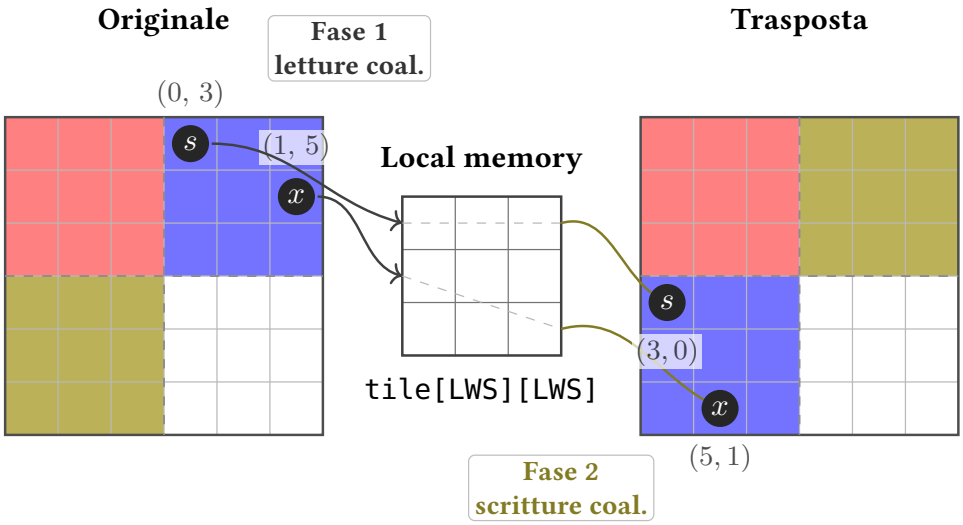

Figura 5.2: Trasposizione con tile in local memory. Si tracciano due elementi del blocco blu: s $(0, 3)$ (diagonale del tile, $l_c = l_r = 0$) e x $(1, 5)$ (fuori diagonale, $l_r = 1$, $l_c = 2$). Le frecce scure mostrano la *Fase 1* (letture coalescenti dall'originale al tile); le linee tratteggiate il percorso interno al tile (orizzontale per s, diagonale per x — lo scambio $r \leftrightarrow c$ avviene qui); le frecce dorate la *Fase 2* (scritture coalescenti verso la trasposta). Le coordinate (r, c) si scambiano: $(0, 3) \rightarrow (3, 0)$ e $(1, 5) \rightarrow (5, 1)$.

```
/* Richiede orig_rows e orig_cols multipli di lws */
kernel void transpose_lmem_k(global int * restrict trasposta,
                             global const int * restrict originale,
                             int orig_rows, int orig_cols,
                             local int * restrict tile)
{
    int li_c = get_local_id(0);
    int li_r = get_local_id(1);
    int lws  = get_local_size(0); /* work-group quadrato */

    int orig_c = get_group_id(0) * lws + li_c;
    int orig_r = get_group_id(1) * lws + li_r;

    /* Fase 1: caricamento con letture coalescenti */
    tile[li_r * lws + li_c] = originale[orig_r * orig_cols + orig_c];

    barrier(CLK_LOCAL_MEM_FENCE);

    /* Fase 2: work-group trasposto, indici locali scambiati */
    int trans_r = get_group_id(0) * lws + li_r;
    int trans_c = get_group_id(1) * lws + li_c;

    trasposta[trans_r * orig_rows + trans_c] = tile[li_c * lws + li_r];
}
```

Listing 5.13: Kernel di trasposizione con tile in local memory

Le coordinate del work-group sono scambiate tra le due fasi: `get_group_id(0)` contribuisce

all'indice di riga nella fase 2 anziché di colonna come nella fase 1. Gli indici locali (l_c, l_r) mantengono invece la loro direzione rispetto agli assi. Se `get_group_id()` non fosse disponibile, si ricava come `get_global_id(d) / get_local_size(d)`.

Lato host la differenza rispetto agli altri kernel è la griglia bidimensionale e l'allocazione del tile come local memory:

```
const size_t lws[2] = { lws_cli, lws_cli }; /* work-group quadrato */
const size_t gws[2] = { round_mul_up(orig_cols, lws[0]),
                        round_mul_up(orig_rows, lws[1]) };

cl_uint arg = 0;
clSetKernelArg(k, arg++, sizeof(cl_mem), &trasposta);
clSetKernelArg(k, arg++, sizeof(cl_mem), &originale);
clSetKernelArg(k, arg++, sizeof(cl_int), &orig_rows);
clSetKernelArg(k, arg++, sizeof(cl_int), &orig_cols);
clSetKernelArg(k, arg++, lws_cli * lws_cli * sizeof(cl_int), NULL);

clEnqueueNDRangeKernel(que, k, 2, NULL, gws, lws, ...);

/* benchmark: lettura originale + scrittura trasposta = 2 * memsize */
printf("transpose: %.4gGB/s\n", 2.0 * memsize / (double)ns);
```

Listing 5.14: Lancio del kernel di trasposizione con tile

5.16 Bank conflict nella local memory

La local memory è fisicamente realizzata con *banchi* di memoria su chip: strutture di 32 bit (un intero) indirizzate in modo *trasversale*, ovvero indirizzi consecutivi cadono su banchi consecutivi. In questo schema ogni colonna del tile (con indice fisico c) corrisponde al banco $c \bmod N_b$, dove N_b è il numero di banchi (tipicamente 32).

Accessi ottimali Work-item diversi dello stesso sottogruppo che accedono a *banchi diversi* vengono serviti in un unico ciclo di clock.

Bank conflict Se due work-item dello stesso sottogruppo richiedono lo *stesso banco*, gli accessi vengono serializzati: il banco risponde a un work-item alla volta, con un ciclo di clock aggiuntivo per ogni conflitto.

Broadcast Eccezione: se tutti i work-item leggono lo stesso identico indirizzo, la local memory invia il dato in broadcast senza alcun conflitto.

Analizziamo il kernel `transpose_lmem_k` con `lws` $= 32$. Nella *fase 1* l'indice fisico è `tile[li_r*lws+li_c]`: al variare di `li_c` il banco vale $(\texttt{li_r} \cdot 32 + \texttt{li_c}) \bmod 32 = \texttt{li_c}$, quindi ogni work-item accede a un banco diverso (nessun conflitto). Nella *fase 2* l'indice diventa `tile[li_c*lws+li_r]`: al variare di `li_c` il banco vale $(\texttt{li_c} \cdot 32 + \texttt{li_r}) \bmod 32 = \texttt{li_r}$, costante per tutti i work-item. Tutti accedono allo stesso banco `li_r`: **bank conflict**.

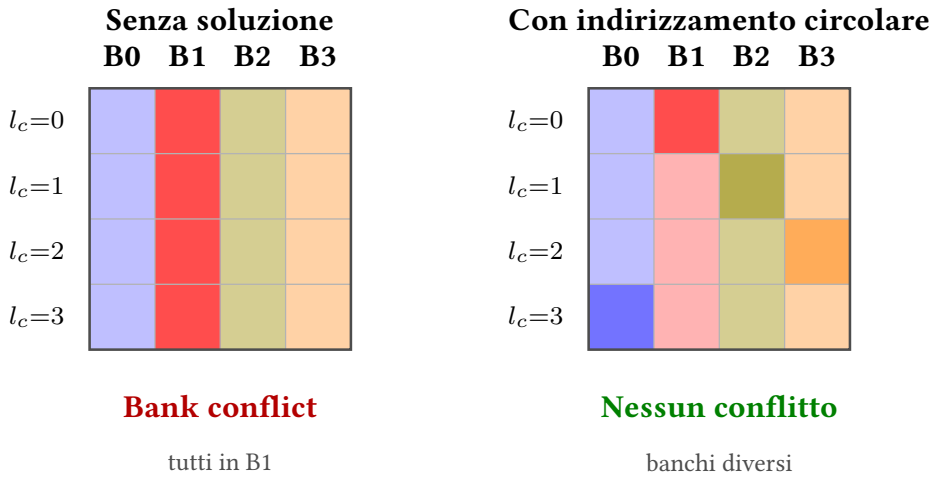

Figura 5.3: Bank conflict in fase 2 di `transpose_lmem_k` (tile 4×4, $l_r = 1$ fissato). Sinistra: senza soluzione, tutti e quattro i work-item leggono dalla colonna B1 $\rightarrow$ serializzazione. Destra: con indirizzamento circolare, ogni work-item accede a un banco diverso $\rightarrow$ accessi paralleli.

5.16.1 Soluzione 1: padding

L'idea è allocare il tile con una colonna extra di *padding* in modo che elementi della stessa colonna concettuale finiscano su banchi diversi.

Lato host. Si alloca `(lws+1)*lws` interi invece di `lws*lws`:

```
clSetKernelArg(k, arg++, (lws_cli + 1) * lws_cli * sizeof(cl_int), NULL);
```

Listing 5.15: Allocazione del tile con padding

Nel kernel. Si introduce `local_pitch = lws + 1` come passo di riga:

```
int local_pitch = lws + 1;   /* colonna extra di padding */

/* Fase 1: letture coalescenti */
tile[li_r * local_pitch + li_c] = originale[orig_r * orig_cols + orig_c];
barrier(CLK_LOCAL_MEM_FENCE);

/* Fase 2: scritture coalescenti */
trasposta[trans_r * orig_rows + trans_c] = tile[li_c * local_pitch + li_r];
```

Listing 5.16: Accesso al tile con padding

Il banco di lettura in fase 2 diventa $(\texttt{li_c} \cdot (\texttt{lws} + 1) + \texttt{li_r}) \bmod 32$. Con $\texttt{lws} = 32$ si ha $\texttt{lws} + 1 = 33$ e $\gcd(33, 32) = 1$: al variare di `li_c` i bank index assumono tutti valori distinti, eliminando il conflitto.

Il principale svantaggio di questa soluzione è l'overhead di memoria: anziché `lws*lws` interi se ne allocano `(lws+1)*lws`, con un incremento di circa il 3% per `lws=32`. La local memory aggiuntiva riduce il numero di work-group che possono coesistere sulla stessa compute unit (occupancy), limitando la capacità dell'hardware di mascherare le latenze con altri work-group attivi. La semplicità di implementazione è invece il punto di forza: sostituire `lws` con `local_pitch` in due righe è immediato e non richiede di ragionare sul layout interno del tile.

5.16.2 Soluzione 2: indirizzamento circolare

La colonna fisica in cui viene scritto un elemento è shiftata circolarmente in base alla riga, senza allocare memoria aggiuntiva:

```
int circular_index = li_c + li_r;
if (circular_index >= lws) circular_index -= lws; /* equivalente a % lws */
```

Listing 5.17: Calcolo dell'indice circolare

La simmetria `li_c+li_r` = `li_r+li_c` garantisce che lo stesso `circular_index` funzioni sia in fase 1 che in fase 2: l'elemento (l_r, l_c) è archiviato in `tile[l_r*lws+circ]`, e in fase 2 il work-item con `li_c` diverso usa lo stesso `circ` per trovarlo.

```
kernel void transpose_circ_k(global int * restrict trasposta,
                             global const int * restrict originale,
                             int orig_rows, int orig_cols,
                             local int * restrict tile)
{
    int li_c = get_local_id(0);
    int li_r = get_local_id(1);
    int lws  = get_local_size(0);

    int orig_c = get_group_id(0) * lws + li_c;
    int orig_r = get_group_id(1) * lws + li_r;

    int circular_index = li_c + li_r;
    if (circular_index >= lws) circular_index -= lws;

    /* Fase 1: caricamento con letture coalescenti */
    if (orig_r < orig_rows && orig_c < orig_cols)
        tile[li_r * lws + circular_index] = originale[orig_r * orig_cols + orig_c];

    barrier(CLK_LOCAL_MEM_FENCE);

    int trans_r = get_group_id(0) * lws + li_r;
    int trans_c = get_group_id(1) * lws + li_c;

    /* Fase 2: indici work-group trasposti, lettura con indice circolare */
    if (trans_r < orig_cols && trans_c < orig_rows)
        trasposta[trans_r * orig_rows + trans_c] = tile[li_c * lws + circular_index];
}
```

Listing 5.18: Kernel di trasposizione con indirizzamento circolare

Il lato host non cambia rispetto a `transpose_lmem_k`: si allocano `lws*lws` interi. Il banco di lettura in fase 2 vale $(\texttt{li_c} \cdot \texttt{lws} + (\texttt{li_c} + \texttt{li_r}) \bmod \texttt{lws}) \bmod 32$; con $\texttt{lws} = 32$ si riduce a $(\texttt{li_c} + \texttt{li_r}) \bmod 32$: tutti i bank index sono distinti al variare di `li_c`.

Per capire intuitivamente perché funziona, è utile osservare il layout fisico del tile dopo la fase 1 (figura 5.4). Senza circular shift ogni riga memorizza gli elementi nello stesso ordine: la colonna virtuale l_c coincide sempre con la colonna fisica, che corrisponde al banco l_c. Tutti gli elementi con la stessa colonna virtuale finiscono quindi nello stesso banco, producendo un conflitto in lettura. Con il circular shift, la riga l_r memorizza i propri elementi con uno shift di l_r

posizioni verso destra (modulo `lws`): la colonna virtuale 1, ad esempio, si trova in B1 per $l_r = 0$, in B2 per $l_r = 1$, in B3 per $l_r = 2$ e in B0 per $l_r = 3$. Ogni elemento della stessa colonna virtuale risiede in un banco diverso, così le letture simultanee di fase 2 colpiscono banchi tutti distinti.

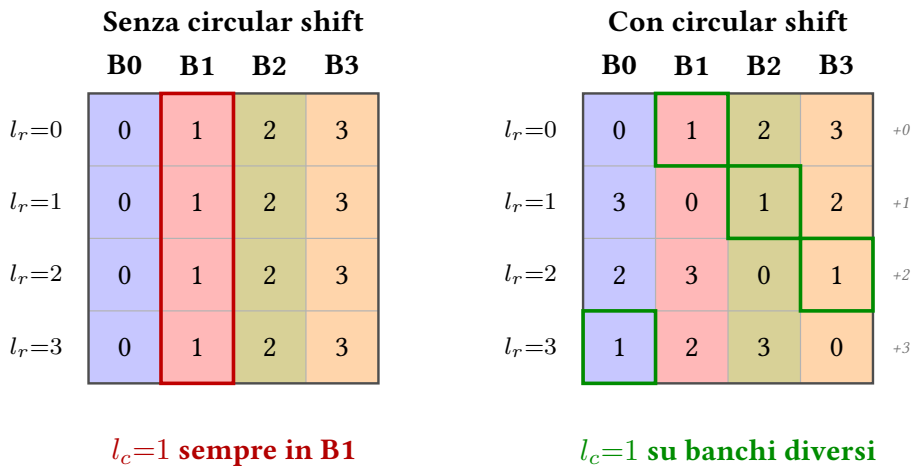

Figura 5.4: Layout fisico del tile 4×4 in fase 1. Ogni cella mostra l'indice di colonna virtuale (l_c) memorizzato in quella posizione fisica. I bordi evidenziati tracciano gli elementi con l_c=1, che vengono letti tutti simultaneamente in fase 2 con l_r=1 fisso. Senza circular shift cadono nella stessa colonna (B1); con circular shift ogni riga applica uno shift crescente (annotazioni *+r* a destra), distribuendo l_c=1 su quattro banchi distinti.

A differenza del padding, l'indirizzamento circolare non richiede alcuna memoria aggiuntiva e lascia inalterata l'occupancy. Il calcolo di `circular_index` introduce una sola addizione e un confronto per work-item, un costo trascurabile rispetto ai guadagni. Le prestazioni risultanti sono molto vicine a quelle del kernel `init`.

Capitolo 6

Immagini in OpenCL

Le *immagini* sono un meccanismo di OpenCL per gestire dati multidimensionali progettato specificamente per le immagini digitali. A differenza dei buffer, non tutto l'hardware supporta la scrittura su immagini nei kernel, e quando lo fa è possibile solo per determinati formati.

6.1 Formato dell'immagine

Alla creazione si specificano due componenti:

Data type Definisce quanti bit sono disponibili per ogni canale e come il dato è codificato:

- `CL_UNSIGNED_INT8` — intero senza segno a 8 bit per canale, letto come intero.
- `CL_UNORM_INT8` — stesso layout binario, ma il valore letto nel kernel è un `float` normalizzato in $[0.0,\ 1.0]$ ($0 \to 0.0$, $255 \to 1.0$).
- `CL_UNORM_INT_101010` — 10 bit per canale colore, restituito come `float` normalizzato.

Channel order Indica quanti canali sono presenti e come sono interpretati:

- `CL_R` — un solo canale (rosso); il valore è passato come prima componente del vettore letto.
- `CL_INTENSITY` — un solo valore fisico replicato su tutti e quattro i canali in lettura.
- `CL_LUMINANCE` — un solo valore replicato sui tre canali colore (R, G, B); il canale alpha è 1.0.

6.1.1 Lettura nel kernel

Indipendentemente dal channel order scelto, la lettura di un pixel da un'immagine restituisce sempre un vettore a quattro componenti (R, G, B, A). Le componenti non definite dall'order vengono impostate a valori di default (0 o 1 a seconda della specifica).